EPHEC
ÉCOLE SUPÉRIEURE
ENSEIGNEMENT POUR ADULTES

Année académique **2026-2027**

Programmation orientée objet — Java

Recueil des exercices

Tous les modules du cours, dans l'ordre des séances

Ce document a été élaboré avec l'aide de Claude, un assistant d'intelligence artificielle développé par Anthropic. L'enseignant en a conçu le contenu et la progression, l'a relu, corrigé et validé, et en assume la responsabilité.

EPHEC — Bachelier en informatique, niveau 2

Table des matières

Comment travailler ces exercices 5
À quoi sert ce document 5
Les classes de test 5
Laissez toujours votre code compilable 5
Les codes, et où ils mènent 6
Module POO — Classes et objets 7
POO.1 — Une première classe 7
POO.2 — La même classe, encapsulée 8
POO.3 — L'âge n'est pas une donnée à stocker 9
POO.4 — Saisie au clavier, et ce qui arrive aux paramètres 10
POO.5 — Changement de base, et logique testable 11
Module STA — Membres de classe 13
STA.1 — Une constante et un compteur d'instances 13
STA.2 — Un constructeur privé et des fabriques 14
Module TMP — Dates et heures 17
TMP.1 — Mesurer un écart entre deux dates 17
TMP.2 — Les dates sont immuables 18
TMP.3 — Compter des secondes : date ou instant ? 19
TMP.4 — L'arithmétique des mois 20
TMP.5 — `Period` ou `Duration` 21
TMP.6 — Formater et analyser une date 22
Module TAB — Tableaux, boucles et opérateur ternaire 24
TAB.1 — Une matrice carrée 24
TAB.2 — Le triangle de Pascal 25
TAB.3 — L'opérateur ternaire 26
TAB.4 — Des objets qui se connaissent 27
Module ENU — Énumérations et records 29
ENU.1 — Un feu de signalisation 29
ENU.2 — Une température, deux unités 30
ENU.3 — Un jeu de cartes 31
ENU.4 — Un `record` d'étudiant 32
Module ABS — Classes abstraites 34
ABS.1 — Des formes géométriques 34
ABS.2 — Des animaux qui font du bruit 35
ABS.3 — Deux banques, un seul traitement 36
ABS.4 — Une hiérarchie à deux niveaux 37
ABS.5 — Tout écrire, à partir de rien 38
Module ITF — Interfaces 40
ITF.1 — Une interface et deux implémentations 40

ITF.2 — Payer en ligne 41
ITF.3 — Des formes qui se déplacent et tournent 42
ITF.4 — Prévenir sans connaître 43
Module PKG — Packages et visibilité 44
PKG.1 — Ce que voit une classe du même package 44
PKG.2 — Ce que voit une classe d'un autre package 45
PKG.3 — Réduire la surface publique d'une classe 46
PKG.4 — Découper un programme en packages 46
PKG.5 — `protected`, ou ce que l'héritage ouvre 47
Module COL — Collections 49
COL.1 — Deux façons d'être trié 49
COL.2 — Un annuaire 50
COL.3 — Lire des noms et les trier 51
COL.4 — Compter les mots d'un texte 52
COL.5 — Filtrer avec un prédicat 52
COL.6 — Un ensemble sans doublons 53
COL.7 — Une table de collections 54
COL.8 — Trois ordres pour une bibliothèque 55
Module EXC — Exceptions 57
EXC.1 — Lever sa propre exception 57
EXC.2 — Un compte, sans exception 58
EXC.3 — Un numéro de compte vérifié 59
EXC.4 — Provoquer, puis traiter 60
EXC.5 — Lire une trace 61
Module JUN — Tests unitaires avec JUnit 63
JUN.1 — Le test comme spécification 63
JUN.2 — Détecter un défaut 64
JUN.3 — Écrire une classe de test 65
Module GEN — Types génériques 67
GEN.1 — Une paire, un seul type 67
GEN.2 — Un couple, deux types 68
GEN.3 — Deux sortes de triplets 69
GEN.4 — Le joker 69
GEN.5 — Une méthode générique 70
GEN.6 — Hériter d'une classe générique 71
Module LAM — Lambdas et références de méthodes 73
LAM.1 — Un tableau de comportements 73
LAM.2 — Écrire son interface fonctionnelle 74
LAM.3 — Celle du JDK, et les références de méthodes 75
Module FX — Interfaces graphiques avec JavaFX 77
FX.1 — Un compteur de participants 78

FX.2 — Un bouton Reset 79
FX.3 — Deux compteurs, et une mise en page 79
FX.4 — Compter au clavier 80
FX.5 — Un événement voyage 81
FX.6 — Écouter une propriété 81
FX.7 — Un Slider et un Label 82
FX.8 — Lier plutôt qu'écouter 83
FX.9 — Une calculatrice, en MVC 83
Module FIC — Fichiers 86
FIC.1 — Trois classes qui se connaissent 86
FIC.2 — Enregistrer des objets 87
FIC.3 — Enregistrer du texte 88

Comment travailler ces exercices

À quoi sert ce document

Chaque fascicule d'exercices commence par la liste de son module. Ce qui suivait cette liste — comment lancer les tests, comment lire un test vert, pourquoi laisser son code compilable — était identique dans les quinze.

C'est ici désormais, une fois pour toutes. Lisez ces quelques pages en début d'année, et revenez-y le jour où quelque chose vous surprend.

Les classes de test

Elles sont déjà écrites, sous `src/test/java`. Lancez-les avec la double flèche verte dans la marge. Tout part rouge : votre objectif est le vert.

Les squelettes lèvent une `UnsupportedOperationException` : remplacez ces lignes par votre code. Ne changez ni les noms, ni les types. Quand une méthode ou un constructeur est déjà déclaré, complétez son corps plutôt que de le régénérer.

Un test vert ne garantit pas que la réponse est correcte. Il vérifie le comportement, pas la lisibilité ni la pertinence de la conception — qui comptent aussi à l'examen.

Une cinquantaine de tests sont verts dès le départ

Ce n'est pas une erreur. Ils relèvent de trois situations, qu'il vaut la peine de savoir distinguer.

1. Un test qui garde une contrainte

Une classe qui doit rester abstraite, un champ qui doit rester privé, une constante qui doit rester finale. Le squelette les respecte déjà.

Ces tests servent le jour où, bloqué par un message, on est tenté de retirer le mot qui gêne. Supprimer `abstract` fait disparaître « cannot be instantiated » — et casse tout ce que l'exercice montrait. Le test rouge vous le dira avant le correcteur.

2. Un exercice où TOUS les tests sont verts au départ

Il y en a trois dans le recueil, et ils ne demandent pas d'écrire du code : ils demandent d'en *transformer*. Réduire une surface publique, découper un programme en packages, écrire une classe de test.

Les tests verts y sont un **filet**, pas un objectif : ils prouvent que votre transformation n'a rien cassé. C'est leur passage au **rouge** qui est le signal.

Retenez-en que le vert n'est pas toujours une cible. Il arrive qu'il soit une garantie.

3. Un test qui vérifie ce que le compilateur a écrit

Les accesseurs, le `toString` et le `equals` d'un `record`. Vous ne les avez pas écrits : le compilateur les a écrits, parce que vous avez déclaré un record.

Les voir verts, c'est constater de ses yeux ce qu'une seule ligne de déclaration vous a évité d'écrire.

Laissez toujours votre code compilable

Tout le dossier `src/main/java` se compile d'un seul bloc. Un exercice laissé à moitié écrit empêche donc de lancer *tous* les autres, y compris ceux des séances précédentes.

Prenez-en une habitude, et non une contrainte. Avant de passer à autre chose, posez-vous la question : est-ce que ça compile ? Si oui, tant mieux. Si non, deux issues — vous corrigez, ou vous mettez le morceau en chantier entre commentaires avec `Ctrl+/` / `⌘/`. Dans les deux cas, vous repartez d'un état sain.

C'est aussi ce que l'examen demande : on y rend du code qui compile. Autant que le réflexe soit pris dès janvier. Voir `ERR.9` du guide IntelliJ si vous vous retrouvez bloqué.

L'exception qui confirme la règle

Un seul exercice du recueil, `ABS.5`, vous livre sa classe de test **entièrement en commentaire**. Elle appelle des classes que vous n'avez pas encore écrites, et elle bloquerait tout le module Maven.

Vous la décommenterez une fois vos classes en place. C'est précisément parce que la règle ci-dessus existe que cet exercice doit s'en écarter.

Les codes, et où ils mènent

Vous rencontrerez trois familles de codes dans les fascicules. Chacune renvoie à un document différent.

Code	Exemple	Où le chercher
`THM.n`	`ABS.3`	un exercice, dans le fascicule de son module
`CC-XXX.n`	`CC-NOM.7`	une règle du mémento Clean Code
`TEC-XXX.n`	`TEC-MVN.4`	une notion du mémento technique
`IDE.n / ERR.n`	`ERR.9`	une manœuvre ou une panne, dans le guide IntelliJ

La numérotation des exercices est thématique, pas chronologique. `EXC.3` est le troisième exercice du module Exceptions, quelle que soit la séance où il tombe. Un exercice garde donc son code même si le calendrier change.

Chaque module s'ouvre sur le tableau de ses exercices. Leurs codes sont **cliquables** : ils mènent directement à l'exercice, comme les lignes de la table des matières.

Module POO — Classes et objets

Les exercices du module. Leurs codes sont **cliquables** : ils mènent directement à l'exercice.

Exercice	Package	Sujet
POO.1	`be.ephec.java.poo.e01`	Une première classe, sans encapsulation
POO.2	`be.ephec.java.poo.e02`	La même classe, encapsulée
POO.3	`be.ephec.java.poo.e03`	L'âge n'est pas une donnée à stocker
POO.4	`be.ephec.java.poo.e04`	Saisie au clavier et passage de paramètres
POO.5	`be.ephec.java.poo.e05`	Conversion de base, et logique testable

POO.1 — Une première classe

Package : `be.ephec.java.poo.e01`

Objectif

Écrire une classe, l'instancier, appeler ses méthodes. Et constater au passage ce qu'implique un champ public.

Énoncé

Créez une classe `Personne` qui modélise une personne par son état civil minimal.

Elle présente quatre champs, tous `public` pour cet exercice-ci : un nom, un prénom, une année de naissance et un âge.

Elle présente aussi deux méthodes :

- `computeAge(int annee)` calcule et retourne l'âge qu'a la personne à l'année passée en paramètre. La méthode fait une simple soustraction, sans traitement particulier.
- `display()` affiche les propriétés de la personne sur la console. Le format d'affichage est libre.

Écrivez ensuite une classe `Programme` contenant une méthode `main` dans laquelle vous créez une personne, renseignez ses champs, puis affichez son prénom, son nom et son âge.

Spécifications à respecter

Élément	Signature	Rôle
`Personne`	`public class`	Classe à écrire, quatre champs public
`nom`	`public String`	Nom de famille
`prenom`	`public String`	Prénom
`anneeDeNaissance`	`public int`	Année de naissance, sur quatre chiffres
`age`	`public int`	Âge — voir la question ci-dessous
`computeAge`	`public int computeAge(int annee)`	Retourne `annee` moins l'année de naissance
`display`	`public void display()`	Affiche la personne sur la console

Comment vérifier votre code

> `be.ephec.java.poo.e01.PersonneTest`
>
> Vérifie que les quatre champs existent et sont bien public, et que `computeAge` retourne le bon écart d'années — y compris pour une année antérieure à la naissance, où le résultat est négatif.

Questions à se poser

- La classe ne déclare aucun constructeur. Pourquoi `new Personne()` fonctionne-t-il quand même, et que valent les champs juste après ?
- Le champ `age` et la méthode `computeAge` donnent tous deux l'âge. Que se passe-t-il si l'un est mis à jour et pas l'autre ? Lequel des deux garderiez-vous ?
- Qu'est-ce qui empêche, dans cette version, d'écrire `personne.anneeDeNaissance = -3000` ?

Lisez les noms de la classe à voix haute. Y en a-t-il un que vous ne sauriez pas prononcer, ou qu'il faudrait épeler à un collègue ? Voir `CC-NOM.3`.

Quand un essai ne marche pas, le réflexe est de mettre l'ancienne version en commentaire « au cas où ». Que devient ce commentaire trois séances plus tard ? Voir `CC-COM.2` — et notez que les corrigés du cours enfreignent cette règle exprès, pour vous montrer ce qui ne compile pas.

Les noms des champs de cette classe sont donnés : `nom`, `prenom`, `anneeDeNaissance`. Vous choisirez les vôtres dès l'exercice suivant — le mémento Clean Code commence en vous donnant huit règles de nommage, `CC-NOM.1` à `CC-NOM.8`. Lisez-les une fois : elles vous serviront toute l'année, ainsi qu'à l'examen et dans votre vie professionnelle.

POO.2 — La même classe, encapsulée

Package : `be.ephec.java.poo.e02`

Objectif

Corriger le défaut de `POO.1` : reprendre le contrôle sur l'état de l'objet.

Énoncé

Reprenez la classe `Personne` de `POO.1` et encapsulez-la : tous les champs deviennent `private`, et l'accès depuis l'extérieur passe par des accesseurs.

Écrivez un getter et un setter pour le nom, le prénom et l'année de naissance : ces trois-là ne posent pas de difficulté.

Le champ `age` demande en revanche une décision, et c'est le vrai travail de cet exercice. Vous avez trois possibilités.

- Un getter et un setter, comme pour les autres champs.
- Un getter seul, sans moyen d'écrire l'âge depuis l'extérieur.
- Ni l'un ni l'autre.

Chacune a une conséquence. Avec un setter, n'importe qui peut écrire un âge sans aucun rapport avec l'année de naissance, et l'objet se retrouve dans un état incohérent. Sans setter, l'âge reste à sa valeur initiale — c'est-à-dire `0` si personne ne l'a renseigné à la construction. Et sans accesseur du tout, le champ devient invisible de l'extérieur : à quoi sert-il encore ?

Tranchez, implémentez votre choix, et notez votre raisonnement en une ou deux phrases. L'exercice suivant reviendra sur cette question, et vous pourrez comparer.

Les méthodes `computeAge` et `display` ne changent pas. Adaptez le `Programme` pour qu'il fonctionne à nouveau.

Note : les accesseurs de `nom`, `prenom` et `anneeDeNaissance` sont déjà déclarés dans le squelette — il ne vous reste qu'à compléter leur corps. Ne les régénérez pas avec `Alt+Inser`, vous obtiendriez des doublons.

Exécutez le `Programme` une fois que tout est vert, et regardez l'âge affiché. Il vaut `0`. Notez pourquoi : la réponse fait le sujet de l'exercice suivant.

Spécifications à respecter

Élément	Signature	Rôle
`nom`	`private String`	+ `getNom()` et `setNom(String)`
`prenom`	`private String`	+ `getPrenom()` et `setPrenom(String)`
`anneeDeNaissance`	`private int`	+ `getAnneeDeNaissance()` et `setAnneeDeNaissance(int)`
`age`	`private int`	Accesseurs : à vous de décider, et de justifier
`computeAge`	`public int computeAge(int annee)`	Inchangée
`display`	`public void display()`	Inchangée

Comment vérifier votre code

> `be.ephec.java.poo.e02.PersonneTest`
>
> Vérifie qu'aucun champ n'est resté accessible de l'extérieur, que chaque setter est bien relu par son getter, et que `computeAge` s'appuie sur l'année réellement enregistrée.

Questions à se poser

- Dans `setNom(String nom)`, que se passerait-il si vous écriviez `nom = nom` au lieu de `this.nom = nom` ? Essayez : le compilateur ne dit rien.
- Un getter et un setter qui ne font que lire et écrire le champ : à quoi servent-ils, puisqu'ils ne contrôlent rien ? Où placeriez-vous une vérification de l'année de naissance pour éviter une naissance en -3000 ?
- Supposons que vous exposiez un setter pour l'âge. Qu'est-ce qui empêche alors d'enregistrer une personne née en 1990 et âgée de 7 ans ?
- Supposons que vous n'en exposiez pas. Qui renseigne l'âge, et que vaut-il tant que personne ne l'a fait ?
- La question de fond : ce champ mérite-t-il d'exister ? Gardez votre réponse sous la main.

Sur le fond de cette question, le mémento Clean Code a deux règles : `CC-OBJ.2` — un objet expose des comportements, pas ses données — et `CC-OBJ.1`, qui demande de parler à un objet plutôt que de l'interroger. Elles éclairent le choix que vous êtes en train de faire.

POO.3 — L'âge n'est pas une donnée à stocker

Package : ***`be.ephec.java.poo.e03`***

Objectif

Repérer une donnée redondante et s'en débarrasser. Découvrir qu'une méthode peut calculer ce qu'un champ prétendait mémoriser.

Contexte

Depuis `POO.1`, votre classe `Personne` contient deux champs qui disent la même chose : `anneeDeNaissance` et `age`. Mais l'un est stable et l'autre ne l'est pas.

L'année de naissance d'une personne ne changera jamais. Son âge, lui, augmente chaque année — et rien dans votre classe ne le met à jour. Le champ `age` est donc juste au moment où on le remplit, puis devient faux et le reste. Une donnée qui se périme toute seule n'a rien à faire dans un objet quand on peut la recalculer.

Énoncé

Reprenez la classe `Personne` de `POO.2` dans le package de cet exercice, et faites-lui subir trois modifications.

- Supprimez le champ `age` ainsi que les accesseurs que vous lui aviez éventuellement donnés en `POO.2`.

- Laissez `computeAge(int annee)` telle quelle : elle retourne l'âge à une année donnée, et cela reste utile.
- Ajoutez une méthode `getAge()`, sans paramètre, qui retourne l'âge de la personne aujourd'hui. Elle ne refait pas le calcul elle-même : elle appelle `computeAge` en lui passant l'année courante.

Pour obtenir l'année courante, utilisez `LocalDate.now().getYear()` (cela implique l'ajout d'un `import java.time.LocalDate`). Adaptez enfin le `Programme` pour qu'il affiche l'âge sans jamais le renseigner.

Note : les classes de gestion du temps du JDK — `LocalDate`, `LocalTime`, `LocalDateTime` — font l'objet du cours suivant. Ici, vous n'avez besoin que de `now()` et de `getYear()`.

Spécifications à respecter

Élément	Signature	Rôle
`nom`	`private String`	+ `getNom()` et `setNom(String)`
`prenom`	`private String`	+ `getPrenom()` et `setPrenom(String)`
`anneeDeNaissance`	`private int`	+ `getAnneeDeNaissance()` et `setAnneeDeNaissance(int)`
`age`	— supprimé —	Le champ et son getter disparaissent
`computeAge`	`public int computeAge(int annee)`	Inchangée : l'âge à l'année passée en paramètre
`getAge`	`public int getAge()`	L'âge aujourd'hui ; délègue à `computeAge`
`display`	`public void display()`	Affiche la personne, âge compris

Comment vérifier votre code

> `be.ephec.java.poo.e03.PersonneTest`
>
> Vérifie qu'aucun champ `age` ne subsiste, que `computeAge` répond toujours correctement, et que `getAge` retourne bien l'âge à l'année courante — sans que personne ne l'ait renseigné.
>
> Remarquez que le test porte surtout sur `computeAge`, qui est déterministe. `getAge` dépend de la date du jour : il est testable, mais seulement de façon indirecte. C'est la raison pour laquelle les deux méthodes coexistent au lieu d'être fondues en une seule.

Questions à se poser

- `getAge()` n'a ni champ derrière lui, ni setter. Où est passée la donnée ?
- `getAge()` ne connaît que l'année de naissance. Une personne née en décembre a-t-elle vraiment, au mois de mars, l'âge que votre méthode annonce ? Que faudrait-il connaître pour répondre exactement — et qu'utiliserez-vous au cours suivant pour le faire ?
- Si vous supprimiez `computeAge` pour ne garder que `getAge()`, que deviendrait la classe de test ?

Ce que vous venez de faire porte un nom : supprimer une donnée dérivée pour la calculer. `CC-CLS.1` en donne la raison générale — une classe doit avoir peu de raisons de changer, et deux données qui disent la même chose en donnent une de trop.

POO.4 — Saisie au clavier, et ce qui arrive aux paramètres

Package : *`be.ephec.java.poo.e04`*

Objectif

Lire des données au clavier avec `Scanner`, et comprendre ce que Java transmet réellement lors d'un appel de méthode.

Énoncé — première partie

Écrivez une classe `Hello` contenant une méthode `main`. Dans cette méthode, utilisez un `Scanner` pour demander à l'utilisateur le nom, le prénom et l'année de naissance d'une personne.

Créez ensuite une `Personne` — une version complète vous est fournie dans le package de cet exercice — renseignez-la avec les valeurs saisies, et affichez-la.

Énoncé — seconde partie

La classe fournie contient déjà une méthode `modifie(int nombre, Personne personne)` qui modifie ses deux paramètres. Complétez le `main` comme suit :

```
int nombre = 5;
System.out.println("avant : " + nombre + " / " + personne.getPrenom());

modifie(nombre, personne);

System.out.println("après : " + nombre + " / " + personne.getPrenom());
```

Exécutez, puis expliquez par écrit ce que vous observez. Les deux paramètres sont traités de la même façon par Java ; pourtant le résultat n'est pas le même pour les deux.

Comment vérifier votre code

> **Pas de classe de test pour cet exercice.**
>
> Un programme qui lit au clavier et écrit à l'écran ne s'automatise pas simplement : il n'expose aucune valeur de retour sur laquelle porter une assertion. C'est exactement le problème que `POO.5` va vous apprendre à éviter.
>
> Vérifiez donc à la main, et gardez votre explication écrite : elle sera discutée en classe juste après.

Pièges connus

- Après un `nextInt()`, le passage à la ligne reste dans le tampon. Un `nextLine()` qui suit retourne alors une chaîne vide. Ordonnez vos questions pour éviter le problème, ou consommez la fin de ligne.
- N'ouvrez qu'un seul `Scanner` sur `System.in` pour tout le programme.

POO.5 — Changement de base, et logique testable

Package : ***`be.ephec.java.poo.e05`***

Objectif

Lire la Javadoc d'une méthode du JDK pour en découvrir une variante. Et séparer pour la première fois le calcul du dialogue avec l'utilisateur.

Contexte

La méthode `Integer.parseInt(String)` que vous connaissez interprète toujours son argument en base 10. Elle possède une seconde version, `Integer.parseInt(String, int)`, dont le second paramètre — appelé `radix` — indique la base depuis laquelle interpréter le nombre.

Les constantes `Character.MIN_RADIX` et `Character.MAX_RADIX` donnent les bases utilisables. Consultez la Javadoc pour connaître leurs valeurs et le comportement de la méthode face à une entrée invalide. Pour l'afficher sans quitter IntelliJ, voir `IDE.14`.

Énoncé

Écrivez une classe `Base` qui contient deux choses bien distinctes.

- Une méthode `public static int enBase10(String nombre, int base)` qui retourne la valeur décimale du nombre reçu. Cette méthode ne lit rien au clavier et n'affiche rien : elle reçoit ses données en paramètres et retourne un résultat.
- Une méthode `main` qui annonce d'abord ce que le programme fait — avec un exemple — puis **boucle** : elle demande le nombre, puis la base, appelle `enBase10`, affiche le résultat, et recommence jusqu'à ce que l'utilisateur entre `-1` comme nombre.

Pourquoi une boucle. Convertir un seul nombre puis quitter oblige à relancer le programme pour en essayer un autre. Or c'est en essayant plusieurs bases sur le même nombre qu'on comprend ce que fait la conversion.

Un piège vous attend. Mélanger `nextInt` et `nextLine` dans une boucle fait sauter une question sur deux : `nextInt` laisse la fin de ligne dans le tampon, et le `nextLine` du tour suivant la ramasse et retourne une chaîne vide.

Essayez, constatez, puis cherchez comment l'*éviter* plutôt que le rattraper. Une solution tient en un mot.

Si l'utilisateur tape quelque chose d'invalide — `FF` en base 10, ou une base de 1 — le programme s'arrêtera brutalement sur une `NumberFormatException` — la boucle comprise, alors qu'il aurait suffi de redemander. C'est le comportement attendu à ce stade : on ne sait pas encore attraper une exception.

Mais la boucle rend le défaut visible, et c'est exactement ce qui motive le module `EXC` — où cet exercice revient sous la forme `EXC.4`.

Spécifications à respecter

Élément	Signature	Rôle
`enBase10`	`public static int enBase10(String, int)`	Retourne la valeur en base 10 ; toute la logique tient ici
`main`	`public static void main(String[])`	Dialogue avec l'utilisateur, et rien d'autre

Exemples attendus

```
enBase10("255",      10)   ->   255
enBase10("1010",      2)   ->    10
enBase10("11111111",  2)   ->   255
enBase10("FF",       16)   ->   255
enBase10("ff",       16)   ->   255     (la casse est sans importance)
enBase10("ABC",      16)   ->  2748
enBase10("777",       8)   ->   511
enBase10("-FF",      16)   ->  -255     (le signe est accepté)
```

Comment vérifier votre code

> `be.ephec.java.poo.e05.BaseTest`
>
> Vérifie les conversions ci-dessus : bases 2, 8, 10 et 16, majuscules comme minuscules, et nombre négatif.
>
> Remarquez que ce test n'est possible que parce que `enBase10` retourne une valeur. Si vous aviez tout écrit dans le main, il n'y aurait rien à vérifier automatiquement. Nous appliquerons cette séparation systématiquement à partir de maintenant.

Questions à se poser

- Faut-il vraiment écrire l'algorithme de conversion vous-même, ou le JDK le fait-il déjà ?
- Pourquoi la base maximale est-elle 36 ?
- `enBase10` ne rattrape pas l'exception : elle la laisse passer. Est-ce un oubli ? Qui est le mieux placé pour décider quoi faire d'une saisie invalide, la méthode de calcul ou le programme qui lance le calcul ?

Cette séparation entre le calcul et le dialogue est la règle `CC-MET.8` du mémento Clean Code : une méthode fait quelque chose ou répond à une question, pas les deux. C'est aussi ce qui rend `enBase10` testable, alors qu'une méthode qui affiche ne l'est pas — voir `CC-TST.1`.

Module STA — Membres de classe

Les exercices du module. Leurs codes sont **cliquables** : ils mènent directement à l'exercice.

Exercice	Package	Sujet
STA.1	`be.ephec.java.sta.e01`	Une constante et un compteur d'instances
STA.2	`be.ephec.java.sta.e02`	Un constructeur privé et des fabriques

Ce que `static` veut dire

Jusqu'ici, tous vos champs appartenaient à un objet : chaque `Personne` a son `nom`, chaque `Compte` son `solde`. Trois personnes, trois cases en mémoire.

Certaines informations n'appartiennent à **aucun** objet en particulier. Par exemple « combien de réservations ont été émises ? ». La question porte sur l'ensemble des objets. C'est ce que `static` exprime.

```
private int numero;           une case PAR OBJET
private static int emises;    UNE case, partagee par tous
```

Trois usages, trois raisons

Forme	Ce qu'elle exprime
`public static final`	une constante : même valeur pour tous les objets, et elle ne change pas
`private static`	un état partagé : il appartient à la classe, pas à un exemplaire
`public static` + méthode	une question qui porte sur l'ensemble, ou une fabrique

`static` et `final` disent deux choses différentes. `static`, que la valeur est partagée ; `final`, qu'elle ne change pas. Une constante a besoin des deux, un compteur a besoin du premier seulement.

Une méthode statique n'a pas de `this`

C'est la source d'erreur la plus fréquente du module. Une méthode statique n'est appelée sur **aucun** objet : `this` n'y existe donc pas, et le compilateur le dit franchement.

```
non-static variable this cannot be referenced from a static context
```

C'est aussi la raison pour laquelle le `main` est statique : il s'exécute avant qu'aucun objet n'existe.

STA.1 — Une constante et un compteur d'instances

Package : *`be.ephec.java.sta.e01`*

Objectif

Distinguer ce qui appartient à l'objet de ce qui appartient à la classe, et le vérifier dans le débogueur.

Contexte

Une salle de spectacle émet des réservations. Chacune porte un nom et un numéro, et les numéros se suivent — la première porte le numéro 1.

Le constructeur ne reçoit pas le numéro. C'est la classe qui le distribue, ce qui garantit qu'aucun ne se répète. Un appelant distrait ne peut plus se tromper.

Énoncé

Complétez `Reservation` :

- la constante `PLACES_MAX` et le compteur vous sont **déclarés**. Une méthode à compléter peut exister sans être écrite : elle lève une `UnsupportedOperationException` en attendant. Un champ, lui, existe ou n'existe pas. Sans ces deux déclarations, la classe de test ne compilerait pas.
- la valeur de la constante, elle, est à corriger ;
- lisez leurs modificateurs et demandez-vous ce que chacun apporte : `public static final` pour l'une, `private static` pour l'autre. Pourquoi pas `final` sur le compteur ?
- le constructeur, qui incrémente le compteur et en tire le numéro ;
- `nombreEmises()` et `resteDeLaPlace()`, qui répondent sur l'ensemble ;
- un `toString()` retournant `Reservation n3 : Grace Hopper`.

À faire dans le débogueur

Placez un point d'arrêt dans le constructeur et créez trois réservations (voir `IDE.19` du guide IntelliJ). À chaque arrêt, avancez avec `F8` jusqu'à la fin du constructeur, puis regardez le panneau Variables :

- dépliez `this` : vous y trouvez `nom` et `numero`, et le numéro diffère d'un objet à l'autre ;
- `emises` n'est pas dans `this` : il est affiché à part, parce qu'il n'appartient pas à l'objet mais à la classe. Sa valeur est la même pour toutes les réservations - il n'y a qu'une case. S'il n'apparaît pas, tapez `Reservation.emises` dans la zone « Evaluate expression ».

> **L'erreur à commettre exprès**
>
> Écrivez `return this.numero;` dans `nombreEmises()`. Lisez le refus du compilateur : il explique tout le module en une phrase.

Comment vérifier votre code

```
nombreEmises() avant toute création        0
numéro de la première réservation          1
numéro de la troisième                     3
nombreEmises() après trois créations       3
toString() de la troisième                 Reservation n3 : Grace Hopper
resteDeLaPlace()                           true
```

Questions à se poser

- Que se passe-t-il si vous incrémentez le compteur *après* avoir affecté le numéro ? Essayez.
- Le compteur est privé. Qu'est-ce que cela empêche exactement, puisque personne ne pourrait le remettre à zéro de toute façon ? Voir `CC-OBJ.1`.
- Le champ partagé rend les tests dépendants de leur ordre : la classe de test le dit et l'assume. Est-ce acceptable ? Voir `CC-TST.3`.
- Pourquoi `PLACES_MAX` s'écrit-il en majuscules, alors que Java n'y oblige pas ? Voir `CC-NOM.7`.

STA.2 — Un constructeur privé et des fabriques

Package : `be.ephec.java.sta.e02`

Objectif

Découvrir qu'une méthode statique peut construire des objets, et pourquoi c'est parfois préférable à un constructeur.

Le problème que l'exercice résout

Une durée s'exprime en minutes. Mais quand on écrit :

```
new Duree(2)
```

deux quoi ? Heures ? Minutes ? Un constructeur ne peut pas le dire : il porte le nom de sa classe, et rien d'autre. Une méthode, elle, porte le nom qu'on veut.

```
Duree.deHeures(2)      aucun doute
Duree.deMinutes(2)     aucun doute non plus
```

Ces méthodes s'appellent des **fabriques**. Deux fabriques peuvent avoir la même signature là où deux constructeurs ne le pourraient pas — elles se distinguent par leur nom.

Ce qui vous est fourni

Le champ `minutes` et le constructeur, déclaré `private`. Personne ne peut donc écrire `new Duree(...)` depuis l'extérieur : c'est ce qui oblige à passer par les fabriques.

Énoncé

Méthode	Ce qu'elle fait
`deMinutes(int)`	fabrique une durée, exprimée en minutes
`deHeures(int)`	fabrique une durée, exprimée en heures
`enMinutes()`	retourne le nombre total de minutes
`plus(Duree)`	retourne une NOUVELLE durée, somme des deux
`total(Duree...)`	additionne un nombre quelconque de durées
`toString()`	« 3h35 », ou « 45min » en dessous d'une heure

Les trois points de `total(Duree...)` signifient « zéro, une ou plusieurs ». On l'appelle avec `Duree.total(a, b, c)` ou même `Duree.total()`, qui doit retourner une durée nulle.

> **Une contrainte d'écriture**
>
> `plus()` retourne une NOUVELLE durée : ni celle sur laquelle on l'appelle, ni celle qu'on lui passe ne changent.
>
> C'est l'immuabilité rencontrée en `TMP.2` avec `LocalDate`, appliquée cette fois à une classe que vous écrivez. Le champ étant final, vous n'avez de toute façon pas le choix.

Comment vérifier votre code

```
deHeures(2).enMinutes()                  120
deHeures(3).plus(deMinutes(35))          3h35
deMinutes(15)                            15min
deHeures(2)                              2h00
total(3h35, 15min, 3h35)                 7h25
total()                                  0min
après a.plus(b), a vaut toujours         sa valeur initiale
```

Questions à se poser

- Écrivez `new Duree(90)` dans le Programme. Que refuse le compilateur, et qu'est-ce que ce refus vous empêche d'écrire de faux ?
- Pourquoi `total()` est-elle statique, alors que `plus()` ne l'est pas ? Qu'est-ce qui les distingue ?
- Le format « 2h00 » impose deux chiffres pour les minutes. Que donnerait votre code sur une durée de 2h05 si vous l'oubliiez ?
- Pourriez-vous ajouter une fabrique `deSecondes(int)` sans toucher au reste de la classe ? Voir `CC-CLS.1`.

- `deMinutes` et `deHeures` disent leur unité dans leur nom. Qu'auriez-vous fait avec un paramètre booléen `enHeures` ? Voir `CC-MET.5`.

Module TMP — Dates et heures

Les exercices du module. Leurs codes sont **cliquables** : ils mènent directement à l'exercice.

Exercice	Package	Sujet
TMP.1	`be.ephec.java.tmp.e01`	Mesurer un écart entre deux dates
TMP.2	`be.ephec.java.tmp.e02`	Les dates sont immuables
TMP.3	`be.ephec.java.tmp.e03`	Compter des secondes : date ou instant ?
TMP.4	`be.ephec.java.tmp.e04`	L'arithmétique des mois
TMP.5	`be.ephec.java.tmp.e05`	`Period` ou `Duration`
TMP.6	`be.ephec.java.tmp.e06`	Formater et analyser une date

Les classes que vous allez rencontrer

Classe	Ce qu'elle représente	Exemple
`LocalDate`	une date, sans heure	`2027-01-19`
`LocalDateTime`	une date avec une heure	`2027-01-19T18:00`
`Period`	un écart en années, mois et jours	`36 ans, 8 mois, 2 jours`
`Duration`	un écart en heures, minutes et secondes	`3 h 35`
`ChronoUnit`	l'unité dans laquelle mesurer un écart	`DAYS, WEEKS, MONTHS`
`DateTimeFormatter`	la conversion entre date et texte	`19/01/2027`

TMP.1 — Mesurer un écart entre deux dates

Package : ***`be.ephec.java.tmp.e01`***

Objectif

Construire une date, et mesurer l'écart qui la sépare d'une autre dans quatre unités différentes.

Énoncé

Complétez la classe `UtilitaireDates`, qui expose quatre méthodes statiques mesurant le même écart dans quatre unités. Aucune ne lit au clavier ni n'affiche : elles reçoivent deux dates et retournent un nombre.

Écrivez ensuite le `Programme` : il demande l'année, le mois et le jour de naissance, construit la date avec `LocalDate.of(annee, mois, jour)`, et affiche les quatre écarts jusqu'à aujourd'hui.

Spécifications à respecter

Élément	Signature	Rôle
`ecartEnJours`	`public static long ecartEnJours(LocalDate, LocalDate)`	Nombre de jours entiers
`ecartEnSemaines`	`public static long ecartEnSemaines(LocalDate, LocalDate)`	Nombre de semaines entières
`ecartEnMois`	`public static long ecartEnMois(LocalDate, LocalDate)`	Nombre de mois entiers
`ecartEnAnnees`	`public static long ecartEnAnnees(LocalDate, LocalDate)`	Nombre d'années entières

Indice : `java.time.temporal.ChronoUnit`. Cherchez la méthode `between` dans la Javadoc, et regardez comment s'écrit un appel.

Exemples attendus

```
Entre le 17/05/1990 et le 19/01/2027 :

  jours    ->  13396
  semaines ->   1913
  mois     ->    440
  annees   ->     36
```

Comment vérifier votre code

> `be.ephec.java.tmp.e01.UtilitaireDatesTest`
> Vérifie les quatre valeurs ci-dessus, l'écart nul, et un écart mesuré à rebours — qui doit être négatif.

Questions à se poser

- Entre le 1er et le 30 janvier, combien de mois y a-t-il selon votre méthode ? Et selon l'intuition ?
- Pourquoi ces méthodes retournent-elles un long et non un `int` ?

Remarquez les noms : `ecartEnJours` dit l'unité, ce que `ecart` ne dirait pas. C'est la règle `CC-NOM.1` du mémento Clean Code — un nom doit répondre à la question qu'on se pose en le lisant.

TMP.2 — Les dates sont immuables

Package : ***`be.ephec.java.tmp.e02`***

Objectif

Rencontrer une erreur qui ne provoque ni refus du compilateur, ni plantage à l'exécution, ni message d'aucune sorte — et qui donne pourtant un résultat faux.

Avant de coder

Exécutez le `Programme` fourni. Il contient une erreur. Lisez sa sortie et essayez de comprendre avant de continuer.

```
LocalDate rendezVous = LocalDate.of(2027, 1, 19);

System.out.println("Rendez-vous initial : " + rendezVous);

rendezVous.plusWeeks(1);

System.out.println("Apres report d'une semaine : " + rendezVous);
```

Les deux lignes affichent la même date. Pourquoi ?

Énoncé

Complétez la classe `Agenda` avec deux méthodes, puis corrigez le `Programme`.

Élément	Signature	Rôle
`dansUneSemaine`	`public static LocalDate dansUneSemaine(LocalDate)`	La date sept jours plus tard
`reporterDe`	`public static LocalDate reporterDe(LocalDate, int)`	Report d'un nombre de jours ; un nombre négatif recule

Comment vérifier votre code

> `be.ephec.java.tmp.e02.AgendaTest`
>
> Vérifie les reports, y compris ceux qui franchissent un mois ou une année, et surtout que la date reçue en paramètre ressort intacte des deux méthodes.

Ce qu'il faut retenir

Un `LocalDate` est *immuable* : sa valeur est fixée à la construction et ne changera jamais. `plusWeeks` ne peut donc pas modifier l'objet — elle doit en fabriquer un nouveau et le retourner. Sans affectation, ce nouvel objet est jeté aussitôt.

Vous avez déjà rencontré cette mécanique sans forcément la nommer : la méthode `texte.toUpperCase()` ne met pas la chaîne en majuscules, elle en retourne une autre. `String` est immuable, exactement comme `LocalDate`.

Ce n'est pas une gêne, c'est une garantie : vous pouvez passer une date à n'importe quelle méthode sans craindre qu'elle la modifie dans votre dos.

Cette garantie rejoint `CC-MET.7` du mémento Clean Code, qui met en garde contre les effets de bord : une méthode qui modifie discrètement ce qu'on lui confie est une source d'erreurs durables. L'immuabilité les rend impossibles.

TMP.3 — Compter des secondes : date ou instant ?

Package : `be.ephec.java.tmp.e03`

Objectif

Découvrir, en butant dessus, la différence entre une date et un instant.

Contexte

En `TMP.1` vous avez compté les jours écoulés depuis votre naissance. Comptez maintenant les *secondes*. La démarche paraît identique : changer `ChronoUnit.DAYS` en `ChronoUnit.SECONDS` et l'affaire est réglée.

Elle ne l'est pas, et c'est tout l'intérêt de l'exercice.

Énoncé

Élément	Signature	Rôle
`secondesEntre`	`public static long secondesEntre(LocalDateTime, LocalDateTime)`	Secondes écoulées entre deux instants
`secondesDepuis`	`public static long secondesDepuis(LocalDate, LocalDateTime)`	Secondes vécues depuis une naissance
`formater`	`public static String formater(long)`	Groupe les chiffres par trois

L'heure de naissance n'est pas connue : comptez à partir de minuit. Cherchez dans la Javadoc de `LocalDate` la méthode qui fabrique un `LocalDateTime` au début de la journée.

Écrivez ensuite le `Programme`, qui demande la date de naissance et affiche le résultat formaté. Décommentez enfin la ligne signalée dans le `Programme` et lisez le message d'erreur : c'est le point central de l'exercice.

Sur le formatage

Un milliard de secondes écrit d'un trait est illisible. Groupez les chiffres par trois, séparés par une espace ordinaire — celle de la barre d'espacement.

Précisez `Locale.US` dans votre formatage. Sans locale explicite, le séparateur dépend de la langue de la machine, et le JDK français emploie une espace insécable étroite qui ressemble à une espace ordinaire sans en être une. Le test échouerait sur une différence invisible à l'œil.

Exemples attendus

```
secondesEntre(19/01 18:00, 19/01 19:00)          ->          3 600
secondesEntre(19/01 00:00, 20/01 00:00)          ->         86 400
secondesDepuis(17/05/1990, 19/01/2027 18:00)     ->  1 157 479 200

formater(1157479200)                             ->  "1 157 479 200"
```

Comment vérifier votre code

> `be.ephec.java.tmp.e03.CompteurTest`
>
> Vérifie l'heure, la journée, une vie entière, la convention de minuit et le formatage.
>
> Si le troisième test échoue sur `UnsupportedTemporalTypeException: Unsupported unit: Seconds`, ne cherchez pas d'erreur dans votre calcul : c'est la leçon de l'exercice.

Ce qu'il faut retenir

Un `LocalDate` ne représente pas un instant. Il représente une case du calendrier — le 17 mai 1990 — sans heure, sans minute, sans seconde. Lui demander combien de secondes le séparent d'une autre case n'a pas de sens, et le JDK refuse plutôt que d'inventer une réponse.

Pour compter des secondes il faut un `LocalDateTime`, c'est-à-dire une date *et* une heure. `atStartOfDay()` fabrique ce `LocalDateTime` en fixant l'heure à minuit — ce qui est une convention, pas une vérité : personne ne naît à minuit, et votre résultat est donc juste à quelques heures près.

Questions à se poser

- Vous avez répondu en `TMP.1` à la question « pourquoi un long plutôt qu'un `int` ». Le nombre de secondes d'une vie la rend concrète : un `int` s'arrête à 2 147 483 647. Au bout de combien d'années le compteur déborderait-il ?
- L'horloge Unix compte les secondes depuis le 1er janvier 1970 : elle en est aujourd'hui à environ 1,79 milliard. Est-ce plus ou moins que le nombre de secondes que vous avez vécues, et pourquoi ?
- Le résultat de `secondesDepuis` change à chaque exécution du Programme, mais pas dans le test. Comment le test s'y est-il pris ?

TMP.4 — L'arithmétique des mois

Package : ***`be.ephec.java.tmp.e04`***

Objectif

Découvrir que l'addition de mois ne se comporte pas comme celle de jours, et pourquoi aucune autre réponse ne serait meilleure.

Énoncé

Complétez `echeance(LocalDate, int)`, qui retourne la date située un nombre de mois après celle reçue.

Complétez ensuite `memeResultatMoisParMois(LocalDate, int)`, qui retourne `true` si ajouter les mois un par un donne la même chose que les ajouter d'un coup.

Ce que vous allez constater

```
31/01/2027 + 1 mois          ->  28/02/2027
```

```
31/01/2028 + 1 mois           ->  29/02/2028   (annee bissextile)
31/01/2027 + 2 mois           ->  31/03/2027
31/01/2027 + 1 mois + 1 mois  ->  28/03/2027   <-- pas le meme resultat
```

Le 31 février n'existe pas. Java rabote donc au dernier jour disponible du mois d'arrivée. Mais une fois raboté, le 31 est perdu : la seconde addition repart du 28 et non du 31.

Comment vérifier votre code

> `be.ephec.java.tmp.e04.EcheancierTest`
>
> Vérifie les quatre cas ci-dessus, en année ordinaire et bissextile, et le fait qu'une date dont le jour existe dans tous les mois ne pose aucun problème.

Questions à se poser

- Quelle autre réponse Java aurait-il pu donner au 31 janvier plus un mois ? Le 3 mars ? Une exception ? Laquelle vous paraîtrait la moins mauvaise ?
- Une échéance de facture à trente jours et une échéance à un mois : tombent-elles le même jour ? Toujours ?
- Dans quel cas vaut-il mieux compter en jours qu'en mois ?

TMP.5 — `Period` ou `Duration`

Package : ***`be.ephec.java.tmp.e05`***

Objectif

Choisir entre deux classes qui mesurent toutes deux un écart, mais pas dans la même unité de pensée.

Contexte

`Period` compte en années, mois et jours : des unités du *calendrier*, dont la longueur varie. Un mois n'a pas toujours trente jours, une année pas toujours 365.

`Duration` compte en heures, minutes et secondes : des unités de *temps*, toujours identiques.

D'où la règle : `Period` pour des dates, `Duration` pour des heures.

Énoncé

Élément	Signature	Rôle
`age`	`public static Period age(LocalDate, LocalDate)`	L'âge en années, mois et jours
`dureeDePartie`	`public static Duration dureeDePartie(LocalDateTime, LocalDateTime)`	La durée écoulée
`dureeLisible`	`public static String dureeLisible(LocalDateTime, LocalDateTime)`	La même durée écrite « 3h35 »

Attention à la dernière. `toHours()` retourne le nombre total d'heures, mais `toMinutes()` retourne le nombre total de minutes — 215, pas 35. Cherchez dans la Javadoc de `Duration` la méthode qui donne seulement les minutes restantes.

Exemples attendus

```
age(17/05/1990, 19/01/2027)        ->  36 ans, 8 mois, 2 jours
duree(19/01 18:00, 19/01 21:35)    ->  215 minutes, soit 3 heures
dureeLisible(meme intervalle)      ->  "3h35"
```

Comment vérifier votre code

> `be.ephec.java.tmp.e05.StatistiquesTest`
> Vérifie la décomposition de l'âge, les deux lectures de la durée, et le format « 3h35 ».

Questions à se poser

- Pourquoi `Period.between` refuse-t-elle deux `LocalDateTime`, et `Duration.between` deux `LocalDate` ?
- Un `Period` de « 1 mois » ajouté au 31 janvier, puis au 15 mars : combien de jours dans chaque cas ? Que dit cela sur ce qu'un `Period` mémorise réellement ?

TMP.6 — Formater et analyser une date

Package : ***`be.ephec.java.tmp.e06`***

Objectif

Passer d'une date à du texte et inversement — et découvrir que l'analyse est bien plus tolérante que vous ne le souhaiteriez.

Énoncé

Élément	Signature	Rôle
`enTexte`	`public static String enTexte(LocalDate)`	Retourne « 19/01/2027 »
`depuisTexte`	`public static LocalDate depuisTexte(String)`	Analyse le même format
`depuisTexteStrict`	`public static LocalDate depuisTexteStrict(String)`	Idem, mais refuse les dates impossibles

Indice : `DateTimeFormatter.ofPattern("dd/MM/yyyy")`, puis `date.format(...)` et `LocalDate.parse(texte, ...)`.

Le piège

Analysez la chaîne « 31/02/2027 » avec le formateur ordinaire, et regardez ce que vous obtenez.

```
depuisTexte("31/02/2027")   ->  2027-02-28
```

Aucune exception, aucun avertissement. Le 31 février n'existe pas, mais le formateur travaille par défaut en mode indulgent et rabote au dernier jour du mois. Une saisie absurde vient de devenir une date parfaitement plausible, et plus rien dans le programme ne permettra de s'en apercevoir.

Pour `depuisTexteStrict`, exigez le mode strict : `withResolverStyle(ResolverStyle.STRICT)`. Ce mode réclame au passage le motif `uuuu` au lieu de `yyyy` — sans quoi il refuse de fonctionner, pour une raison que la Javadoc explique.

Comment vérifier votre code

> `be.ephec.java.tmp.e06.FormatBelgeTest`
> Vérifie le formatage dans les deux sens, le refus d'un texte mal formé, le rabotage silencieux du 31 février par le mode ordinaire, et son refus par le mode strict.

Questions à se poser

- Vous écrivez un formulaire d'inscription. Lequel des deux modes utilisez-vous, et pourquoi ?
- Le mode ordinaire n'est pas un défaut du JDK : dans quel cas préférerait-on qu'une date soit ajustée plutôt que refusée ?

- Que retourne `depuisTexte("19-01-2027")` ? Et pourquoi ce cas-là est-il refusé alors que le 31 février passe ?

Le mode indulgent illustre `CC-ERR.1` du mémento Clean Code : mieux vaut une exception qu'un code de retour ou qu'une valeur arrangée. Une date rabotée sans avertissement est exactement la valeur arrangée que la règle proscrit.

Module TAB — Tableaux, boucles et opérateur ternaire

Les exercices du module. Leurs codes sont **cliquables** : ils mènent directement à l'exercice.

Exercice	Package	Sujet
TAB.1	be.ephec.java.tab.e01	Une matrice carrée
TAB.2	be.ephec.java.tab.e02	Le triangle de Pascal
TAB.3	be.ephec.java.tab.e03	L'opérateur ternaire
TAB.4	be.ephec.java.tab.e04	Des objets qui se connaissent

Ce qu'il faut avoir en tête

Un tableau a une taille *fixe*, décidée au moment du `new` et jamais modifiable ensuite. C'est sa limite, et c'est aussi ce qui le rend simple et rapide. Les collections, qui grandissent toutes seules, viendront au module `COL`.

```
int[]   nombres = new int[5];              // 5 cases, toutes a 0
String[] mots   = { "Hello", "world" };     // 2 cases, deja remplies

int[][] matrice = new int[3][3];           // un tableau DE TABLEAUX
matrice[1]                                 // la deuxieme ligne, un int[]
matrice[1][2]                              // la troisieme case de cette ligne
```

Retenez surtout la dernière ligne du bloc. Un tableau à deux dimensions n'existe pas vraiment en Java : c'est un tableau dont chaque case contient un autre tableau. Cela a une conséquence que `TAB.2` exploite — les lignes n'ont pas toutes à faire la même longueur.

TAB.1 — Une matrice carrée

Package : ***be.ephec.java.tab.e01***

Objectif

Parcourir un tableau à deux dimensions, et séparer une bonne fois le calcul de l'affichage.

Énoncé

Écrivez la classe `Matrice`, qui expose deux méthodes statiques. Aucune des deux n'affiche quoi que ce soit.

Élément	Signature	Rôle
carree	`public static int[][] carree(int n)`	
enTexte	`public static String enTexte(int[][])`	La matrice sous forme de texte

Le format d'`enTexte` est imposé, parce que le test le vérifie au caractère près : les valeurs d'une même ligne séparées par une espace, les lignes séparées par un retour à la ligne (`"\n"`), et pas de retour à la ligne à la fin.

```
carree(3)             ->  { {1,2,3}, {4,5,6}, {7,8,9} }
enTexte(carree(3))    ->  "1 2 3\n4 5 6\n7 8 9"
carree(0)             ->  un tableau vide
```

Écrivez ensuite le `Programme` : il annonce ce qu'il fait, demande n à l'utilisateur, et affiche la matrice. Il ne devrait lui rester qu'une seule instruction d'affichage.

Une ligne d'annonce suffit — « Ce programme génère une matrice carrée contenant les chiffres de 1 à n^2. » Sans elle, l'utilisateur voit surgir une question sans savoir ce qu'on attend de lui.

Comment vérifier votre code

> `be.ephec.java.tab.e01.MatriceTest`
>
> Vérifie les matrices 1×1, 3×3 et 4×4, le cas d'une taille nulle, et le format exact retourné par `enTexte`.

Questions à se poser

- Vous avez deux boucles imbriquées et un seul compteur qui avance de 1 à n^2. Où faut-il le déclarer, et pourquoi pas à l'intérieur de la boucle ?
- Le séparateur ne doit pas apparaître avant le premier élément ni après le dernier. Est-il plus simple de le placer avant chaque élément sauf le premier, ou après chaque élément sauf le dernier ?
- Que se passerait-il si `enTexte` affichait au lieu de retourner une chaîne ? Le test serait-il encore possible ?

TAB.2 — Le triangle de Pascal

Package : ***`be.ephec.java.tab.e02`***

Objectif

Construire un tableau à deux dimensions dont les lignes n'ont pas toutes la même longueur.

Contexte

Le triangle de Pascal donne les coefficients du binôme de Newton, c'est-à-dire du développement de `(a + b)` élevé à une puissance. Chaque nombre y est la somme des deux qui le surplombent, et les bords valent 1.

```
          1                (a+b)^0
        1   1              (a+b)^1  =  a + b
      1   2   1            (a+b)^2  =  a2 + 2ab + b2
    1   3   3   1          (a+b)^3
  1   4   6   4   1        (a+b)^4
```

Énoncé

Écrivez la classe `Pascal` avec une méthode `public static int[][] triangle(int n)` qui retourne le triangle jusqu'à la ligne n comprise.

Le tableau retourné n'est pas rectangulaire. La ligne d'indice `i` contient `i + 1` nombres. Vous déclarerez donc `new int[n][]` — le nombre de lignes seulement — puis vous créerez chaque ligne à sa propre taille.

Écrivez ensuite le `Programme` : il annonce ce qu'il fait, demande `n`, puis affiche le triangle en utilisant **deux boucles `for-each` imbriquées**. La première parcourt des `int[]`, la seconde des `int`.

Une ligne d'annonce suffit — « Ce programme calcule un triangle de Pascal dont vous spécifiez la taille. » Sans elle, l'utilisateur voit surgir une question sans savoir ce qu'on attend de lui.

Comment vérifier votre code

> `be.ephec.java.tab.e02.PascalTest`
>
> Vérifie le triangle à cinq lignes, la longueur de chaque ligne, la règle de construction sur l'ensemble du triangle, et le cas de zéro ligne.

Questions à se poser

- Les deux bords valent 1. Pour la première ligne, les deux instructions qui les posent écrivent dans la même case. Est-ce un problème ?
- La boucle qui remplit l'intérieur d'une ligne ne tourne pas du tout pour les deux premières lignes. Pourquoi, et est-ce voulu ?

- Une boucle `for-each` permet-elle de modifier le tableau qu'elle parcourt ? Essayez, et regardez ce que dit le compilateur.

TAB.3 — L'opérateur ternaire

Package : ***`be.ephec.java.tab.e03`***

Objectif

Écrire en une instruction ce qui en demanderait cinq avec un `if`, et comprendre où s'arrête l'intérêt de le faire.

Contexte

```
condition ? valeurSiVrai : valeurSiFaux

public int max(int a, int b) {
    return a > b ? a : b;
}
```

La différence essentielle avec le `if` : le ternaire retourne une valeur. On peut donc le placer à droite d'un `return` ou d'une affectation, ce qu'un `if` ne permet pas.

Énoncé

Le package contient un petit jeu de dés, fourni et fonctionnel à deux méthodes près. Deux joueurs lancent un dé à tour de rôle ; celui qui fait le plus grand nombre marque un point, jusqu'à ce que l'un atteigne le maximum.

Complétez les deux méthodes suivantes. Chacune tient en UNE instruction, avec un opérateur ternaire.

Élément	Signature	Rôle
`meilleur`	`public static Player meilleur(Player, Player)`	Celui qui a le plus de points ; en cas d'égalité, le premier
`gagnantDuTour`	`public static Player gagnantDuTour(Player, int, Player, int)`	Celui qui a fait le plus grand nombre ; en cas d'égalité, le premier

Remarquez que ces deux méthodes ne lancent aucun dé. Elles reçoivent les résultats en paramètres. C'est ce qui les rend prévisibles — et donc testables, alors qu'une méthode qui tire au hasard ne le serait pas. Vous retrouverez cette idée au module `JUN`.

Comment vérifier votre code

> `be.ephec.java.tab.e03.GameTest`
>
> Vérifie les deux méthodes, y compris les cas d'égalité — qui doivent retourner le premier joueur. Lancez ensuite le Programme : il joue une vraie partie.

Questions à se poser

- Attention au sens de la comparaison : selon la façon dont vous l'écrivez, l'égalité tombe du côté du premier ou du second joueur. Le test le vérifie.
- Le ternaire peut s'imbriquer : `a ? x : (b ? y : z)`. Écrivez une fois un ternaire à trois niveaux, relisez-le, et décidez si vous le garderiez.
- Le mémento Clean Code n'interdit pas le ternaire, mais `CC-MET.2` demande des méthodes courtes et lisibles. Où placez-vous la limite ?

TAB.4 — Des objets qui se connaissent

Package : `be.ephec.java.tab.e04`

Objectif

Faire collaborer plusieurs classes, et ranger des objets dans un tableau de taille fixe.

Contexte

Un concessionnaire vend des voitures. Chaque vente porte sur une voiture, à un prix catalogue, avec une réduction. Un vendeur conserve la trace de ses ventes.

```
  Person  <---[1]---  Vendor  ---[*]--->  Sale  ---[1]--->  Car
```

Énoncé

Classe	Ce qu'elle contient	Méthodes à écrire
`Car`	marque, modèle, numéro de série — tous final	les trois accesseurs
`Sale`	une voiture, un prix catalogue, une réduction	`prixReel()`, `enTexte()`
`Vendor`	une personne, un tableau de ventes	`enregistrer()`, `nombreDeVentes()`, `chiffreDAffaires()`

Le prix réel ne se stocke pas. Il se déduit du prix catalogue et de la réduction. En faire un champ créerait une donnée redondante qui pourrait se désynchroniser — c'est exactement le problème du champ `age` rencontré en `POO.3`.

Le format d'`enTexte` est imposé :

```
"Renault Clio [VF123] : 20000.0 -> 18000.0"
```

Le tableau de ventes

Un vendeur enregistre au plus `Vendor.MAX_VENTES` ventes, dans un tableau. Un tableau ne grandit pas : sa taille est décidée à la construction. Vous devrez donc retenir vous-même combien de cases sont réellement occupées.

`enregistrer` retourne `true` si la vente a pu être ajoutée, `false` si le tableau est plein.

Comment vérifier votre code

> `be.ephec.java.tab.e04.VenteTest`
>
> Vérifie la voiture, le calcul du prix réel avec et sans réduction, le format exact d'`enTexte`, l'accumulation des ventes et du chiffre d'affaires, et le refus de la onzième vente.

Questions à se poser

- La boucle de `chiffreDAffaires` doit-elle aller jusqu'à `ventes.length` ou jusqu'au nombre de ventes enregistrées ? Essayez la première version et regardez ce qui se passe.
- `Sale` a besoin de la marque et du modèle pour son texte. Vaut-il mieux qu'elle lise les champs de la voiture, ou qu'elle les lui demande ? Voir `CC-OBJ.1` du mémento Clean Code.
- Faut-il un accesseur qui retourne le tableau de ventes ? Que pourrait alors en faire l'appelant ? Voir `CC-OBJ.2`.
- Quels champs de `Vendor` pourraient être `final` ? Le tableau des ventes en fait-il partie, alors que son contenu change ?

Une remarque sur le vendeur

Un `Vendor` possède une `Person` : c'est un champ, comme un autre. Le vendeur est vu ici comme un rôle que joue une personne. On aurait pu écrire, tout aussi légitimement, qu'un vendeur EST une personne : c'est l'héritage, que vous connaissez par le C#. Les deux modèles se défendent ; celui-ci permettrait à la même personne de jouer d'autres rôles, d'être cliente par exemple, sans changer de classe.

Le module `ABS` compare les deux approches sur cet exemple.

Module ENU — Énumérations et records

Les exercices du module. Leurs codes sont **cliquables** : ils mènent directement à l'exercice.

Exercice	Package	Sujet
ENU.1	be.ephec.java.enu.e01	Un feu de signalisation
ENU.2	be.ephec.java.enu.e02	Une température, deux unités
ENU.3	be.ephec.java.enu.e03	Un jeu de cartes
ENU.4	be.ephec.java.enu.e04	Un record d'étudiant

L'énumération en deux minutes

Une `enum` est une classe dont on énumère à l'avance toutes les valeurs possibles. Il n'en existera jamais d'autres, et le compilateur le sait.

```
public enum DayOfWeek { MONDAY, TUESDAY, WEDNESDAY }

DayOfWeek jour = DayOfWeek.TUESDAY;
jour.ordinal()          // 1 : la position dans la declaration
jour.name()             // "TUESDAY"
DayOfWeek.values()      // un tableau des trois valeurs, dans l'ordre
DayOfWeek.valueOf("MONDAY")   // retrouve une constante par son nom
```

L'ordre de déclaration compte. C'est celui que retournent `ordinal()` et `values()`, et `ENU.1` s'en sert pour calculer une couleur suivante sans écrire une seule condition.

Une `enum` est une classe. Elle peut donc contenir des champs, un constructeur et des méthodes, ce qu'`ENU.3` met en pratique.

ENU.1 — Un feu de signalisation

Package : *`be.ephec.java.enu.e01`*

Objectif

Déclarer une énumération, et découvrir que l'ordre des constantes peut remplacer une cascade de conditions.

Énoncé

Écrivez l'énumération `CouleurDeFeu` avec trois constantes : `VERT`, `ORANGE`, `ROUGE` — dans cet ordre, qui est celui du cycle.

Écrivez ensuite la classe `FeuDeSignalisation`. Un feu a une couleur et un état.

Élément	Signature	Rôle
`constructeur`	`public FeuDeSignalisation()`	Un feu neuf est inactif, donc orange
`desactive`	`public void desactive()`	Inactif et orange ; ne changera plus de couleur
`active`	`public void active()`	Actif et rouge
`changeDeCouleur`	`public void changeDeCouleur()`	VERT → ORANGE → ROUGE → VERT ; sans effet si inactif
`getCouleur`	`public CouleurDeFeu getCouleur()`	La couleur actuelle
`estActif`	`public boolean estActif()`	L'état actuel

Élément	Signature	Rôle
`enTexte`	`public String enTexte()`	« ROUGE (actif) » ou « ORANGE (inactif) »

Écrivez enfin le `Programme` : créez un feu, tentez de le faire changer alors qu'il est inactif, activez-le, faites-le changer plusieurs fois, puis désactivez-le.

Comment vérifier votre code

> `be.ephec.java.enu.e01.FeuDeSignalisationTest`
>
> Vérifie l'ordre des trois constantes, l'état d'un feu neuf, l'immobilité d'un feu inactif, le cycle complet des couleurs et le format d'`enTexte`.

Questions à se poser

- Le constructeur doit mettre le feu dans le même état que `desactive()`. Faut-il recopier les instructions, ou appeler la méthode ?
- **Le vrai enjeu de l'exercice.** Vous pouvez écrire `changeDeCouleur` avec trois `if` ou un `switch`. Vous pouvez aussi ne rien tester du tout, en vous servant de `values()`, d'`ordinal()` et du reste de la division par 3. Écrivez les deux, et comparez ce qui se passerait si le feu comptait une quatrième couleur.
- Comparez avec trois constantes `int` valant 0, 1 et 2. Qu'est-ce qu'une énumération empêche d'écrire qu'un `int` autoriserait ? Voir `CC-NOM.7` et `CC-NOM.1`.
- Dans la version sans test, d'où vient le 3 ? C'est le nombre de couleurs, écrit en dur : avec une quatrième couleur, il faudrait penser à le changer aussi. Remplacez-le par `CouleurDeFeu.values().length` — un tableau a une longueur, pas de `size()`, réservé aux collections — et vérifiez qu'ajouter une couleur ne demande plus que sa déclaration. Voir `CC-NOM.7`.

ENU.2 — Une température, deux unités

Package : ***`be.ephec.java.enu.e02`***

Objectif

Se servir d'une énumération comme paramètre, et retomber sur la question de la donnée redondante.

Énoncé

Écrivez l'énumération `TemperatureUnit` avec `CELSIUS` et `FAHRENHEIT`, puis la classe `Temperature`.

Élément	Signature	Rôle
`constructeur`	`public Temperature(double, TemperatureUnit)`	La valeur, et l'unité dans laquelle elle est exprimée
`getValue`	`public double getValue(TemperatureUnit)`	La température dans l'unité demandée
`setValue`	`public void setValue(double, TemperatureUnit)`	Remplace la température

Les formules

```
Celsius     vers Fahrenheit :  f = c * 9 / 5 + 32       (10 C  ->   50 F)
Fahrenheit  vers Celsius    :  c = (f - 32) * 5 / 9     (23 F  ->   -5 C)
```

> **Le piège**
>
> Si vos conversions retournent des valeurs fausses, regardez le type des opérations. 9 / 5 vaut 1 en Java, parce que ce sont deux entiers et que la division entière tronque le résultat. Il suffit qu'un seul des opérandes soit un double pour que le calcul devienne décimal.
>
> Le message d'échec du test vous le rappellera, mais autant le savoir avant.

Comment vérifier votre code

> `be.ephec.java.enu.e02.TemperatureTest`
>
> Vérifie les deux conversions du cours, les repères classiques — 0 °C, l'ébullition, et −40 qui est la seule température identique dans les deux unités — et le comportement de `setValue`.

Questions à se poser

- Combien de valeurs votre objet stocke-t-il ? Si vous en gardez deux, une en Celsius et une en Fahrenheit, que se passe-t-il si l'une est modifiée et pas l'autre ?
- Vous avez choisi une unité de référence pour le stockage. Un utilisateur de la classe peut-il savoir laquelle ? Est-ce souhaitable ?
- L'énumération pourrait contenir elle-même les conversions, puisqu'une enum est une classe. Ce serait-il mieux ? Vous aurez les moyens de trancher après `ENU.3`.
- Les deux méthodes de conversion portent des noms opposés. Le vocabulaire reste-t-il cohérent d'une méthode à l'autre ? Voir `CC-MET.4`.

ENU.3 — Un jeu de cartes

Package : ***`be.ephec.java.enu.e03`***

Objectif

Donner à une énumération un constructeur et des méthodes, et construire un tableau à partir de deux énumérations.

Une enum est une classe

Elle peut donc contenir un champ, un constructeur et des méthodes. Chaque constante énumérée passe ses arguments à ce constructeur.

```
public enum Discount {
    BASIC(0.05), FIDELITY(0.075), SPECIAL(0.10);

    private final double taux;

    Discount(double taux) { this.taux = taux; }

    public double appliquer(double prix) { return prix * (1 - taux); }
}
```

Trois détails de syntaxe : la liste des constantes se termine par un point-virgule dès qu'il y a autre chose après elle, le constructeur est implicitement privé — on ne peut pas faire de `new` sur une enum — et le champ est final.

Énoncé

Écrivez `Color` et `Value`, deux énumérations dont chaque constante connaît son libellé français : `PIQUE` connaît `"Pique"`, `AS` connaît `"As"`, `DIX` connaît `"10"`. Les valeurs se déclarent dans l'ordre du jeu, de l'as au roi.

Écrivez ensuite `Card`, dont le `toString` retourne `"As de Pique"` ou `"Roi de Trèfle"`, puis `Deck` :

- son constructeur crée les 52 cartes. Servez-vous de `Color.values()` et `Value.values()` : deux boucles `for-each` imbriquées suffisent, et le nombre 52 n'a pas à apparaître dans votre calcul ;
- `carteA(int)` retourne la carte à une position ;
- `shuffle()` mélange le paquet. Une façon de faire : parcourir le tableau de la fin vers le début et, à chaque étape, échanger la carte courante avec une carte tirée au hasard parmi elle-même et celles qui la précèdent — `hasard.nextInt(i + 1)`. Les cartes déjà parcourues sont à leur place définitive.

Le programme

Créez un jeu, affichez les 52 cartes **à la suite** et séparées par des virgules — et non une par ligne. Mélangez, puis affichez-les à nouveau.

Le piège est le séparateur. Où placer la virgule sans en laisser une à la fin ? On s'en tire d'ordinaire par un test sur l'indice, qui encombre la boucle. Cherchez `StringJoiner` dans la documentation : on lui donne le séparateur, on ajoute les éléments, et il les place *entre* eux.

Le gain n'est pas la longueur du code, c'est qu'il n'y a plus de condition à lire — donc plus d'endroit où se tromper.

Comment vérifier votre code

> `be.ephec.java.enu.e03.DeckTest`
>
> Vérifie la taille des deux énumérations, les libellés, le texte des cartes, la présence de 52 cartes toutes différentes, et que le mélange ne perd ni ne duplique aucune carte tout en changeant réellement l'ordre.

Questions à se poser

- La carte ne sait pas comment s'écrit une couleur en français : elle le demande à l'énumération. Où aurait-il fallu mettre cette information si Color n'avait pas pu contenir de champ ?
- Pourquoi `nextInt(i + 1)` et non `nextInt(52)` ? Les deux mélangent, mais un seul le fait équitablement. Cherchez « mélange de Fisher-Yates » si la question vous intéresse.
- Deux cartes `"As de Pique"` construites séparément sont-elles égales ? Testez avec `==`, et gardez la question de côté : elle reviendra.
- L'ordre de déclaration des valeurs décide du résultat de `compareTo`. Est-ce visible en lisant la classe, ou faut-il le deviner ? Voir `CC-COM.3` sur ce qu'une Javadoc doit apporter.

ENU.4 — Un `record` d'étudiant

Package : ***`be.ephec.java.enu.e04`***

Objectif

Écrire en une ligne ce qui en demanderait trente, et savoir quand ce n'est pas approprié.

Contexte

Un record modélise une donnée simple et immuable. Le compilateur écrit pour vous le constructeur, les accesseurs, `toString`, equals et `hashCode`.

```
public record Point(int x, int y) { }

Point origine = new Point(0, 0);
origine.x()          // l'accesseur s'appelle x(), pas getX()
origine.toString()   // "Point[x=0, y=0]", genere par le compilateur
```

Un record est final et ses composants aussi. Rien ne peut en hériter, et rien ne peut être modifié après construction. C'est ce qui le rend adapté à une donnée, et inadapté dès qu'un objet porte une logique métier ou doit changer d'état.

Énoncé

L'en-tête du record `Student` vous est fourni : un `nom` (`String`) et une `cote` (`int`), dans cet ordre. Regardez-le bien — cette seule ligne suffit au compilateur pour écrire le constructeur, les accesseurs, `toString`, `equals` et `hashCode`.

Votre part commence à `passe()`, qui retourne `true` si la cote atteint 10.

Écrivez ensuite le `Programme` : créez un **tableau** d'étudiants et affichez chacun d'eux suivi de `"reussi"` ou `"echec"`.

Comment vérifier votre code

> `be.ephec.java.enu.e04.StudentTest`
>
> Les trois premiers tests vérifient le travail du COMPILATEUR : les deux composants et leur ordre, les accesseurs sans préfixe get, le `toString` et le equals générés. Vous n'avez rien écrit de tout cela.
>
> Le dernier vérifie le vôtre : la limite de réussite à 10.

Questions à se poser

- Écrivez la même chose sous forme de classe ordinaire, avec ses champs, son constructeur, ses accesseurs et son `toString`. Combien de lignes ?
- Rien n'empêche de construire un `Student` avec une cote de 47. Où placeriez-vous une vérification, et qu'est-ce qui devrait se produire ? Vous aurez la réponse au module `EXC`.
- `passe()` n'est ni un champ ni un accesseur : c'est un calcul. Pourquoi ne pas stocker un booléen `reussi` dans le record ? Voir `POO.3`.
- Revenez aux deux cartes `"As de Pique"` d'`ENU.3`, construites séparément : `==` les disait différentes. Que donnerait `equals` si `Card` était un `record` ? Et pourquoi ?

Ce que vous verrez en lançant les tests

Trois des sept tests seront verts avant que vous n'écriviez une ligne. Ce n'est pas une panne, et c'est même toute la démonstration de l'exercice.

Deux d'entre eux vérifient les accesseurs, le `toString` et le `equals`. Vous ne les avez pas écrits : le compilateur les a écrits, parce que vous avez déclaré un record. Les voir passer, c'est constater de ses yeux ce qu'une seule ligne de déclaration vous a évité.

Le troisième mesure la forme du `record`. Les quatre autres sont rouges : un par cote essayée, tous sur la seule chose que le compilateur ne pouvait pas deviner : `passe()`, la règle de réussite.

Écrivez la même classe sans record, pour comparer. Deux champs, un constructeur, deux accesseurs, un `toString`, un `equals`, un `hashCode`. Comptez les lignes, puis relisez la déclaration du `record`.

- Un `record` ne fournit ni setter ni constructeur vide. Qu'est-ce que cela vous interdit, et pourquoi est-ce souhaitable ici ? Voir `CC-OBJ.2`.

Module ABS — Classes abstraites

Les exercices du module. Leurs codes sont **cliquables** : ils mènent directement à l'exercice.

Exercice	Package	Sujet
ABS.1	`be.ephec.java.abs.e01`	Des formes géométriques
ABS.2	`be.ephec.java.abs.e02`	Des animaux qui font du bruit
ABS.3	`be.ephec.java.abs.e03`	Deux banques, un seul traitement
ABS.4	`be.ephec.java.abs.e04`	Une hiérarchie à deux niveaux
ABS.5	`be.ephec.java.abs.e05`	Tout écrire, à partir de rien

Ce qu'il faut avoir en tête

Une classe abstraite ne s'instancie pas. Elle sert de socle : elle contient ce que toutes ses sous-classes ont en commun, et laisse abstrait ce qui les distingue.

```
public abstract class Forme {

    // ce qui VARIE : chaque sous-classe le definit
    public abstract String nom();

    // ce qui est COMMUN : ecrit une seule fois
    public void affiche() {
        System.out.println("Je suis un " + nom());
    }
}
```

La règle de partage. Une méthode dont le code serait identique dans toutes les sous-classes appartient à la classe mère. Une méthode dont le code diffère à chaque fois est abstraite. Le reste du module n'est que l'application de ce principe.

Recopier la même méthode dans trois sous-classes est une erreur très fréquente à l'examen. Voir `CC-CLS.3` du mémento Clean Code, qui la traite précisément.

ABS.1 — Des formes géométriques

Package : ***`be.ephec.java.abs.e01`***

Objectif

Écrire une première classe abstraite, et décider ce qui monte et ce qui descend.

Énoncé

Complétez la classe abstraite `Shape`, qui représente une forme géométrique. Elle connaît la position de son centre — un `Point`, fourni — et une rotation en degrés.

Élément	Signature	Rôle
`nom`	`public abstract String nom()`	« cercle », « carre » ou « triangle »
`affiche`	`public void affiche()`	Écrit « Je suis un cercle et je m'affiche »
`efface`	`public void efface()`	Écrit « Je suis un carre et je m'efface »
`getCentre`	`public Point getCentre()`	Le centre
`getRotation`	`public double getRotation()`	La rotation, toujours entre 0 et 360

Élément	Signature	Rôle
`setRotation`	`public void setRotation(double)`	Fixe la rotation, ramenée dans l'intervalle

Complétez ensuite les trois classes `Circle`, `Square` et `Triangle`. Chacune ne devrait contenir qu'une seule méthode.

Écrivez enfin le `Programme` : un **tableau de `Shape`** contenant les trois formes, parcouru pour les afficher puis les effacer.

Sur la rotation

La rotation vaut toujours entre 0 inclus et 360 exclu. `setRotation(370)` donne 10, `setRotation(360)` donne 0.

> **Un piège de l'arithmétique Java**
>
> En Java, le reste de la division garde le signe du dividende : -90 % 360 vaut -90 et non 270. Un modulo seul ne suffit donc pas à ramener une valeur dans un intervalle positif.
>
> Le test vérifie ce cas. Cherchez comment corriger le résultat en une expression.

Comment vérifier votre code

> `be.ephec.java.abs.e01.ShapeTest`
>
> Vérifie que `Shape` est bien abstraite, le nom de chaque forme, la conservation du centre, la normalisation de la rotation y compris pour les valeurs négatives, et le polymorphisme dans un tableau de `Shape`.

Questions à se poser

- Essayez d'écrire `new Shape(new Point(0, 0))`. Que dit le compilateur, et pourquoi ?
- Le constructeur de `Shape` est déclaré `protected` et non `public`. Qu'est-ce que cela change, sachant qu'on ne peut de toute façon pas instancier la classe ?
- Vos trois sous-classes font-elles plus d'une méthode ? Si oui, qu'est-ce qui aurait dû monter dans `Shape` ?
- Le tableau est déclaré `Shape[]` et ne contient aucun `Shape`. Comment est-ce possible ?

ABS.2 — Des animaux qui font du bruit

Package : ***`be.ephec.java.abs.e02`***

Objectif

Refaire le même geste sur un cas plus court, et voir le polymorphisme à l'œuvre sur un tableau dont on ignore le contenu.

Énoncé

Complétez la classe abstraite `Animal` et ses deux sous-classes `Dog` et `Cat`.

Élément	Signature	Rôle
`cri`	`public abstract String cri()`	« J'aboie » pour un chien, « Je miaule » pour un chat
`makeNoise`	`public void makeNoise()`	Écrit le cri sur la console

Écrivez ensuite le `Programme` : un tableau de dix `Animal`, rempli au hasard avec une chance sur deux d'obtenir un chien ou un chat, puis parcouru en appelant `makeNoise()` sur chaque élément.

Contrainte : aucun test de type. Pas d'`instanceof`, pas de comparaison de classe. C'est le polymorphisme qui doit choisir la bonne méthode — voir `CC-CLS.4` du mémento Clean Code.

Comment vérifier votre code

> `be.ephec.java.abs.e02.AnimalTest`
>
> Vérifie que la classe est abstraite, le cri de chaque animal, et que les éléments d'un tableau d'`Animal` retournent bien le cri de leur type réel.

Questions à se poser

- Ajoutez une `Vache`. Combien de lignes faut-il modifier dans le `Programme` ?
- Refaites l'exercice en supprimant la classe abstraite, avec deux classes indépendantes. Qu'est-ce qui devient impossible ?

ABS.3 — Deux banques, un seul traitement

Package : *`be.ephec.java.abs.e03`*

Objectif

Compléter un dispositif écrit par quelqu'un d'autre, en n'ajoutant que ce qui manque réellement.

Contexte

Une application affiche les mouvements bancaires de ses utilisateurs. Elle fonctionne aujourd'hui avec une seule banque, Fric en Vrac. Il faut y ajouter Banquier Pépère.

Le problème : les deux banques livrent les mêmes informations, mais pas dans le même ordre de colonnes.

```
Fric en Vrac      compte, montant, communication
Banquier Pepere   communication, compte, montant
```

Ce qui vous est fourni

Classe	Rôle
`Mouvement`	Un mouvement : compte, montant, communication
`FricEnVracDataProvider`	Les données brutes de la première banque
`BanquierPepereDataProvider`	Celles de la seconde
`VisualiseurDeDonneesBancaires`	La classe abstraite : conversion, total, rapport
`VisualiseurFricEnVrac`	Le visualiseur de la première banque — votre modèle

La classe abstraite laisse trois choses indéfinies : d'où viennent les données, comment s'appelle la banque, et comment se lit une ligne. Tout le reste — convertir l'ensemble, additionner, mettre en forme — y est déjà écrit.

Énoncé

Complétez `VisualiseurBanquierPepere` en définissant les trois méthodes abstraites, puis écrivez le `Programme` : un tableau contenant les deux visualiseurs, parcouru pour afficher chaque rapport et chaque total.

> **La mesure de la réussite**
>
> Votre classe devrait faire une vingtaine de lignes, dont trois méthodes de deux lignes chacune. Si elle est plus longue, vous avez recopié quelque chose qui existait déjà dans la classe mère.
>
> Et le Programme ne doit contenir ni `instanceof`, ni condition sur la banque : il traite un tableau de `VisualiseurDeDonneesBancaires` sans savoir ce qu'il contient.

Comment vérifier votre code

> `be.ephec.java.abs.e03.VisualiseurTest`
>
> Vérifie d'abord le visualiseur fourni, qui sert de référence, puis le vôtre : le nombre de mouvements, l'ordre des colonnes, le total calculé par la classe mère, la forme du rapport, et le traitement des deux banques dans un même tableau.

Questions à se poser

- Une troisième banque arrive, avec encore un autre ordre de colonnes. Combien de classes faut-il écrire, et combien faut-il en modifier ?
- La méthode qui convertit une ligne est déclarée `protected`. Pourquoi pas `public`, puisque c'est une méthode qu'une autre classe doit définir ?
- Le rapport et le total sont écrits une seule fois, dans la classe mère. Qu'est-ce qui se serait passé le jour d'une correction du calcul si chaque visualiseur avait eu le sien ?

ABS.4 — Une hiérarchie à deux niveaux

Package : ***`be.ephec.java.abs.e04`***

Objectif

Placer chaque donnée et chaque calcul au bon étage d'une hiérarchie, et décider où implémenter une méthode d'interface.

Contexte

Une société de services facture ses prestations de deux façons : au forfait, ou à l'heure. Chaque façon connaît deux variantes.

Les trois exercices précédents n'avaient **qu'un seul** niveau d'abstraction. Celui-ci en a deux, et c'est la forme que prend la question 4 de l'examen.

La structure imposée

```
Facturable          interface : double montant()
 |
 +- Prestation      abstraite : le client, et facture()
     |
     +- Forfait     abstraite : prixForfait
     |   +- Abonnement
     |   +- Installation
     |
     +- Horaire     abstraite : tauxHoraire, heures
         +- Depannage
         +- Urgence
```

Ce que chaque classe concrète ajoute

Classe	Champ supplémentaire	Ce que retourne **`montant()`**
`Abonnement`	—	`prixForfait`
`Installation`	`deplacement` (`double`)	`prixForfait + deplacement`
`Depannage`	—	`heures x tauxHoraire`
`Urgence`	`majoration` (`double`)	`heures x tauxHoraire x (1 + majoration)`

La majoration est une *fraction* : 0.5 pour cinquante pour cent.

Ce qui vous est fourni

L'interface `Facturable` et les signatures des constructeurs. Tout le reste est à écrire.

Deux décisions vous appartiennent

- **Où implémenter `montant()`.** L'interface l'exige, mais ni Prestation, ni Forfait, ni Horaire ne savent calculer. Où la méthode doit-elle donc apparaître ?
- **Quelle visibilité pour les accesseurs intermédiaires.** Le prix du forfait est un détail de calcul dont les sous-classes ont besoin. Doit-il être public ? Voir `PKG.5`.

Le programme

Rangez une prestation de chaque sorte dans un **tableau de Prestation**, parcourez-le, affichez chaque facture et le total.

Si vous écrivez un `instanceof` dans cette boucle, c'est que la hiérarchie est mal découpée. Reprenez-la.

Comment vérifier votre code

```
Abonnement(Ada, 125)                    125.00
Installation(Alan, 200, 45.50)          245.50
Depannage(Grace, 65, 3)                 195.00
Urgence(Charles, 65, 2, 0.5)            195.00
facture() du premier                    Ada : 125.00 EUR
total des quatre                        760.50
```

Questions à se poser

- `Prestation` implémente `Facturable` sans écrire `montant()`. Est-ce légal, et pourquoi le compilateur l'accepte-t-il ?
- La méthode `facture()` appelle `montant()`, qui n'existe nulle part quand on l'écrit. Comment est-ce possible ?
- Ajoutez une cinquième sorte de prestation. Combien de classes existantes devez-vous modifier ?

ABS.5 — Tout écrire, à partir de rien

Package : ***`be.ephec.java.abs.e05`***

Ce qui change, et c'est beaucoup

Cet exercice n'a pas de squelette. C'est le seul du recueil. Les quatre précédents vous donnaient les classes, les signatures et les types : il restait à remplir les corps. Ici, vous partez d'un fichier `Programme` vide.

L'énoncé décrit ce que le programme doit faire. Il ne dit pas combien de classes écrire, ni laquelle est abstraite, ni où placer chaque méthode. Ce sont les décisions de l'exercice, et ce sont celles qu'on vous demandera à l'examen.

> **La classe de test est fournie, mais en commentaire**
>
> Elle appelle des classes qui n'existent pas encore. Un fichier qui ne compile pas empêche la compilation de tout le module — y compris les exercices déjà réussis.
>
> Quand vos classes seront écrites : sélectionnez tout le contenu du fichier et faites Ctrl+/ pour retirer les commentaires d'un coup. Les noms attendus sont rappelés en tête du fichier.

Le contexte

Un service postal facture trois sortes d'envois. Chacun a un destinataire et un poids en grammes.

Sorte	Ce qu'elle coûte
`Lettre`	1,10 € jusqu'à 50 g inclus ; 2,20 € au-delà
`Colis`	4,00 € de forfait, plus 0,005 € par gramme
`Recommandé`	7,00 €, quel que soit le poids

Ce que le programme doit faire

- créer un envoi de chaque sorte ;
- les ranger dans un tableau du **type le plus général possible** ;
- le parcourir en affichant l'étiquette de chacun, au format `Ada Lovelace - 120 g - 4.60 EUR` ;
- afficher le total.

Aucun test de type dans cette boucle. Si vous y écrivez un `instanceof` ou un `switch` sur la sorte d'envoi, la hiérarchie est mal découpée.

Les trois questions à se poser avant d'écrire

- **Qu'est-ce que les trois sortes ont en commun ?** Ce qui est commun monte dans la classe mère, et ne s'écrit qu'une fois.
- **Que sait faire un envoi « en général » ?** S'il y a une chose qu'il ne sait pas faire alors que toutes ses sortes la savent, vous tenez votre méthode abstraite.
- **L'étiquette doit-elle être écrite trois fois ?** Regardez ce qui y change d'une sorte à l'autre. Puis relisez `ABS.1`.

Comment vérifier votre code

```
lettre de 20 g                          1.10
lettre de 50 g                          1.10  (la borne est incluse)
lettre de 51 g                          2.20
colis de 120 g                          4.60
colis de 1000 g                         9.00
recommandé, quel que soit le poids      7.00
étiquette d'un colis de 120 g           Ada Lovelace - 120 g - 4.60 EUR
total des trois exemples                12.70
```

Faites attention à la borne des 50 grammes. « Jusqu'à 50 g » les inclut : c'est `<=` et non `<`. Une erreur d'un gramme ne se voit pas en essayant au hasard, et le test à la borne l'attrape tout de suite.

Questions à se poser

- Le recommandé ignore le poids : sa méthode de calcul ne lit aucun champ. Pourquoi hérite-t-il quand même de la classe mère ?
- Ajoutez une quatrième sorte d'envoi. Combien de classes existantes devez-vous modifier ?
- Comparez le temps passé sur cet exercice et sur `ABS.4`. Où est passée la différence : à écrire, ou à décider ?

Module ITF — Interfaces

Les exercices du module. Leurs codes sont **cliquables** : ils mènent directement à l'exercice.

Exercice	Package	Sujet
ITF.1	`be.ephec.java.itf.e01`	Une interface et deux implémentations
ITF.2	`be.ephec.java.itf.e02`	Payer en ligne
ITF.3	`be.ephec.java.itf.e03`	Des formes qui se déplacent et tournent
ITF.4	`be.ephec.java.itf.e04`	Prévenir sans connaître

Interface ou classe abstraite ?

Les deux permettent de manipuler des objets différents à travers un type commun. La différence tient à ce qu'elles peuvent contenir.

	Classe abstraite	Interface
Contient du code ?	oui, et des champs	des signatures seulement
Combien par classe ?	une seule	autant qu'on veut
Ce qu'elle exprime	« est un »	« sait faire »

Un carré *est une* forme : classe abstraite. Un carré *sait* se déplacer, comme une image ou un texte, qui ne sont pas des formes : interface.

Les interfaces de ces exercices vous sont fournies. Elles tiennent en trois lignes et servent de modèle. Le travail porte sur ce qui compte vraiment : les implémenter, et écrire du code qui dépend de l'interface et non des classes concrètes.

ITF.1 — Une interface et deux implémentations

Package : ***`be.ephec.java.itf.e01`***

Objectif

Le cas le plus simple possible, pour isoler l'idée : une méthode qui reçoit un comportement en paramètre.

Énoncé

L'interface `Calculateur` vous est fournie : elle déclare `int calcule(int a, int b)` et rien d'autre.

- Complétez `Addition` et `Multiplication`, qui l'implémentent.
- Complétez `Programme.effectueCalcul(int a, int b, Calculateur calculateur)`, qui effectue un calcul à l'aide du calculateur reçu.
- Écrivez le `main` : créez les deux objets et effectuez quelques calculs.

Le point important

Regardez le type du troisième paramètre : `Calculateur`, et non `Addition`. La méthode ne sait pas quel calcul elle déclenche, et n'a pas besoin de le savoir.

La conséquence est spectaculaire : elle fonctionnera avec des calculateurs qui n'existaient pas quand elle a été écrite. Le test le démontre en lui passant une soustraction fabriquée sur place.

Comment vérifier votre code

> `be.ephec.java.itf.e01.CalculateurTest`
>
> Vérifie que les deux classes implémentent l'interface, que chacune calcule correctement, que `effectueCalcul` délègue, et qu'elle accepte un calculateur qu'elle n'a jamais vu.

Questions à se poser

- L'interface ne dit ni `public` ni `abstract` devant sa méthode. Ajoutez-les : le compilateur proteste-t-il ? Pourquoi les omet-on ?
- Que se passerait-il si `effectueCalcul` prenait un `Addition` en paramètre ? Écrivez-la ainsi, puis essayez de lui passer une multiplication.

ITF.2 — Payer en ligne

Package : ***`be.ephec.java.itf.e02`***

Objectif

La même idée sur un cas réaliste, où l'interface sépare vraiment deux parties d'un programme.

Énoncé

L'interface `Paiement` est fournie : elle déclare `String effectuerPaiement(double montant)`.

Complétez `PayPal` et `CarteDeCredit`. Chacune retourne le compte-rendu de ses étapes, séparées par un retour à la ligne, sans retour final.

```
PayPal, pour 42.50 :
    Connexion a PayPal
    Selection du moyen de paiement
    Validation de la transaction de 42.50 euros

CarteDeCredit, pour 42.50 :
    Authentification de la transaction
    Paiement de 42.50 euros execute
```

Complétez ensuite `Client.executePaiement(Paiement, double)`, puis écrivez le `Programme` qui fait exécuter un paiement par chaque moyen.

> **Pourquoi retourner du texte plutôt que l'afficher**
>
> Une classe de paiement qui écrirait elle-même sur la console deviendrait inutilisable dans une application graphique, et intestable : on ne vérifie pas ce qui est parti dans la console.
>
> En retournant le compte-rendu, elle laisse l'appelant décider quoi en faire — l'afficher, l'enregistrer, l'envoyer par courriel. C'est la règle `CC-MET.7` du mémento Clean Code.
>
> Précisez `Locale.US` dans votre formatage du montant, sinon une machine configurée en français écrirait « 42,50 » avec une virgule, et le test échouerait sur un seul caractère.

Comment vérifier votre code

> `be.ephec.java.itf.e02.PaiementTest`
>
> Vérifie le texte exact retourné par chaque moyen de paiement, le format du montant à deux décimales, et que le client se contente de transmettre.

Questions à se poser

- Le virement bancaire arrive. Quelles classes faut-il écrire, et lesquelles faut-il modifier ?
- Le `Client` ne connaît ni `PayPal` ni la carte. Pourrait-il les connaître ? Qu'est-ce qu'on y perdrait ?

ITF.3 — Des formes qui se déplacent et tournent

Package : ***`be.ephec.java.itf.e03`***

Objectif

Combiner héritage et interfaces sur les formes d'`ABS.1`, et trancher une question de conception.

Énoncé

Les formes d'`ABS.1` sont reprises ici, complètes. Deux interfaces vous sont fournies : `Translatable`, qui déclare `translate(double dx, double dy)`, et `Rotatable`, qui déclare `rotate(double alpha)`. La classe `Shape` les implémente toutes les deux.

Complétez les deux méthodes. La translation déplace le centre ; la rotation s'ajoute à la rotation actuelle, toujours ramenée entre 0 et 360. Chacune affiche ce qu'elle fait :

```
Je suis un carre et je me deplace de 3.0, 4.0
Je suis un triangle et je tourne de 90.0 degres
```

La question à trancher

Ces deux méthodes doivent-elles rester abstraites dans `Shape`, ou celle-ci peut-elle les définir une fois pour toutes ?

Répondez avant de coder, puis vérifiez : si votre réponse était la bonne, les trois sous-classes n'auront pas une ligne à changer.

Comment vérifier votre code

> `be.ephec.java.itf.e03.ShapeTest`
>
> Vérifie que `Shape` implémente bien les deux interfaces, le déplacement du centre, l'accumulation des translations, la normalisation de la rotation, et que les trois formes se comportent identiquement.
>
> Un dernier test manipule un tableau de `Translatable` : les objets y sont vus à travers l'interface seulement.

Questions à se poser

- Dans le `Programme`, déclarez un tableau de `Translatable` et essayez d'appeler `rotate` puis `nom` sur ses éléments. Que dit le compilateur, et pourquoi est-ce souhaitable ?
- Une image et un texte pourraient aussi être translatables sans être des formes. Cela se ferait-il avec une classe abstraite ?
- La normalisation de l'angle apparaît en `ABS.1` et ici. Est-ce une duplication au sens de `CC-CLS.3` ? Les deux packages étant indépendants, la question mérite d'être posée.

ITF.4 — Prévenir sans connaître

Package : `be.ephec.java.itf.e04`

Objectif

Voir ce qu'une interface rend possible : ajouter un acteur à un programme sans toucher au programme.

Contexte

Un magasin prévient plusieurs personnes quand un produit arrive. Aujourd'hui, un `Client` et un `Employe` sont abonnés, et chacun réagit à sa façon.

Ce dispositif porte un nom, le patron Observateur. Il repose entièrement sur le fait que le magasin ne connaît aucun de ses observateurs : il ne connaît que l'interface qu'ils respectent.

Énoncé

Un `Prospect` veut lui aussi être prévenu. Complétez la classe, dont le message doit valoir exactement :

```
Publicite pour le nouveau grille-pain envoyee a Dupont
```

Écrivez ensuite le `Programme` : créez un magasin, abonnez-y les trois acteurs, réceptionnez un produit, et affichez le dernier message de chacun.

Vous ne modifiez ni le Magasin, ni l'interface, ni les deux autres observateurs. Si vous vous surprenez à devoir le faire, relisez le `Magasin` : rien n'y nomme un `Client` ou un `Employe`.

Comment vérifier votre code

> `be.ephec.java.itf.e04.MagasinTest`
>
> Vérifie que `Prospect` implémente l'interface, son message avant et après une annonce, la diffusion aux trois acteurs en un seul appel, et le remplacement du message à la réception suivante.

Questions à se poser

- Combien de fichiers avez-vous modifiés pour ajouter un troisième acteur ? Combien en auriez-vous modifiés si le `Magasin` avait tenu trois listes séparées, une par type d'acteur ?
- Le magasin appelle `nouveauProduit` sur chaque abonné. Comment sait-il quelle version exécuter ?
- Le tableau du `Magasin` a une taille fixe de dix. Où est la limite de ce montage, et qu'est-ce qui la lèvera au module `COL` ?

Module PKG — Packages et visibilité

Les exercices du module. Leurs codes sont **cliquables** : ils mènent directement à l'exercice.

Exercice	Package	Sujet
PKG.1	be.ephec.java.pkg.e01	Ce que voit une classe du même package
PKG.2	be.ephec.java.pkg.e02	Ce que voit une classe d'un autre package
PKG.3	be.ephec.java.pkg.e03	Réduire la surface publique d'une classe
PKG.4	be.ephec.java.pkg.e04	Découper un programme en packages
PKG.5	be.ephec.java.pkg.e05	protected, ou ce que l'héritage ouvre

Trois niveaux de visibilité, pas deux

La plupart des étudiants connaissent `private` et `public`, et croient que le choix se limite à ces deux-là. Il en existe un troisième, qui n'a pas de mot-clé : quand vous n'écrivez rien devant un membre, il devient visible depuis tout le package, et de là seulement.

Ce que vous écrivez	Qui peut y accéder	Usage typique
private	la classe elle-même, et rien d'autre	champs, méthodes internes
(rien)	toutes les classes du même package	collaboration entre classes proches, classes de test
public	n'importe quelle classe du programme	l'interface offerte au reste du monde

Il existe un quatrième niveau, `protected`, qui n'a de sens qu'avec l'héritage. Vous l'avez rencontré au module `ABS` ; `PKG.5` y revient, maintenant que les packages permettent de voir tout ce qu'il ouvre.

PKG.1 — Ce que voit une classe du même package

Package : `be.ephec.java.pkg.e01`

Objectif

Constater par l'expérience ce que chaque niveau de visibilité autorise, depuis une classe voisine.

Énoncé

La classe `Coffre` vous est fournie. Elle expose six membres, deux par niveau de visibilité :

```
private String codeSecret   = "1234";
        String numero       = "C-07";      // pas de mot-cle
public  String proprietaire = "Ada";

private String ouvrirLeMecanisme()  { ... }
        String verifierLeNumero()   { ... }
public  String decrire()            { ... }
```

La classe `Inspecteur` vit dans le *même* package. Sa méthode `experimenter()` contient six lignes en commentaire, une par membre. Décommentez-les une à une et notez celles que le compilateur refuse.

Complétez ensuite `membresAccessibles()` pour qu'elle rende la liste des membres réellement atteignables, séparés par une virgule et un espace, dans l'ordre de déclaration.

Comment vérifier votre code

> `be.ephec.java.pkg.e01.InspecteurTest`
>
> Compare votre réponse à la liste attendue. Le test ne vous dira pas laquelle est fausse : c'est l'expérimentation qui doit vous donner la réponse, pas le test.

Questions à se poser

- Vous êtes dans une autre classe, et pourtant certains membres privés vous sont refusés alors que d'autres membres passent. Contre quoi `private` protège-t-il exactement : contre l'extérieur de l'objet, ou contre l'extérieur de la classe ?
- Deux objets `Coffre` différents : le premier peut-il lire le `codeSecret` du second ? Essayez.

PKG.2 — Ce que voit une classe d'un autre package

Packages : ***`be.ephec.java.pkg.e02.coffre`*** *et* ***`be.ephec.java.pkg.e02.controle`***

Objectif

Franchir la frontière du package, et comprendre à quoi sert vraiment un accesseur.

Énoncé — première partie

La même classe `Coffre` est maintenant seule dans son package. La classe `ControleExterne` vit ailleurs. Sa méthode `experimenter()` reprend les mêmes lignes commentées qu'en `PKG.1`.

Décommentez, et comparez avec ce que vous aviez obtenu à l'exercice précédent. Un seul élément a changé entre les deux exercices — le package — et pourtant le résultat n'est pas le même. Notez lequel des six membres bascule.

Énoncé — seconde partie

Le contrôle extérieur a besoin de lire le numéro du coffre. Complétez `numeroDu(Coffre)` pour qu'elle le rende.

Contrainte : le champ `numero` doit rester tel qu'il est. Le passer en `public` ferait compiler l'exercice en dix secondes, et c'est précisément ce qu'il faut éviter. Trouvez l'autre solution.

Spécifications à respecter

Élément	Signature	Rôle
`numero`	`String numero`	Visibilité par défaut — ne pas modifier
`(à ajouter)`	`public String getNumero()`	Retourne le numéro, en lecture seule
`numeroDu`	`public String numeroDu(Coffre)`	Utilise l'accesseur depuis l'autre package

Comment vérifier votre code

> `be.ephec.java.pkg.e02.controle.ControleExterneTest`
>
> Vérifie que le numéro se lit depuis l'autre package, et que le champ n'est pas devenu public au passage. Les deux tests doivent passer ensemble : rendre le champ public ferait échouer le second.

Le point important

Un champ `public` donne deux droits à la fois — lire et écrire — à tout le monde et pour toujours. Un getter `public` ne donne que la lecture, et laisse la classe maîtresse de ce qu'elle retourne : elle peut demain calculer la valeur, la formater, ou refuser de la donner.

C'est la réponse à la question que beaucoup se posent en `POO.2` : à quoi bon écrire un getter qui se contente de retourner le champ ? À cela. Le getter et le champ `public` ne sont pas équivalents, même quand leur code se ressemble.

C'est aussi la règle `CC-OBJ.2` du mémento Clean Code, prise par l'autre bout : un objet expose des comportements, pas ses données. Le champ `public` expose une donnée ; l'accesseur expose un comportement, celui de la donner.

PKG.3 — Réduire la surface publique d'une classe

Package : `be.ephec.java.pkg.e03`

Objectif

Décider, membre par membre, ce qui doit être visible — au lieu de tout ouvrir par défaut.

Énoncé

La classe `Deck` fonctionne, mais tout y est `public` : la liste des cartes, le compteur, et jusqu'aux étapes internes de la préparation. N'importe qui peut vider le paquet, remplacer la liste, ou appeler `melanger()` au milieu d'une partie.

Le `Programme` fourni définit ce dont un utilisateur a réellement besoin. Il n'appelle que trois opérations. Réduisez la visibilité de tout le reste, sans modifier le `Programme`, qui doit continuer à compiler.

Comment choisir

- `private` si personne d'autre que la classe n'en a besoin — c'est le cas le plus fréquent.
- **visibilité par défaut** si une classe voisine du même package doit y accéder. C'est le cas de `verifierIntegrite()`, que la classe de test appelle.
- `public` seulement pour ce que le `Programme` utilise.

Comment vérifier votre code

> `be.ephec.java.pkg.e03.DeckTest`
>
> Vérifie qu'aucun champ n'est resté public, que seules les trois opérations utilisées le sont encore, et que le paquet fonctionne toujours.

Questions à se poser

- La classe de test est dans le même package que `Deck`. Est-ce un hasard ? Que se passerait-il si elle était ailleurs ?
- Rendre un membre `public` est une promesse : celle de ne pas le changer, puisque d'autres s'en servent. Combien de promesses la version d'origine faisait-elle ?

Le mémento Clean Code aborde la même idée sous l'angle de la taille : `CC-CLS.2` demande des classes courtes, et une classe se mesure autant à ce qu'elle expose qu'à ce qu'elle contient.

PKG.4 — Découper un programme en packages

Package : `be.ephec.java.pkg.e04`

Objectif

Organiser un programme en couches, et voir dans quel sens vont les dépendances.

Énoncé

Quatre classes cohabitent aujourd'hui dans un seul package. Répartissez-les en trois :

Package cible	Classes	Rôle
`…e04.modele`	`Joueur, Partie`	Les données du domaine
`…e04.service`	`Inscription`	Les opérations métier
`…e04.application`	`Application`	Le point d'entrée

Utilisez le refactoring d'IntelliJ plutôt que le copier-coller. Sélectionnez la classe dans l'arborescence, puis `F6` (Move). IntelliJ déplace le fichier, corrige la déclaration de package et ajoute les `import` nécessaires dans toutes les classes concernées.

Comment vérifier votre code

> `be.ephec.java.pkg.e04.DecoupageTest`
>
> Vérifie que chaque classe se trouve à sa nouvelle place, et qu'aucune ne subsiste à l'ancienne. Ce dernier test vous protège du copier-coller : deux copies d'une même classe est le pire des résultats.

Ce qu'il faut observer

Une fois le découpage fait, regardez les imports. Le modèle n'importe rien. Le service importe le modèle. L'application importe les deux. Les flèches vont toutes dans le même sens :

```
application  ->  service  ->  modele
```

Ce n'est pas un accident, c'est le but. Le modèle ne connaît personne, donc il peut servir ailleurs — dans une application graphique, par exemple. Le jour où une dépendance remonte en sens inverse, c'est le signe qu'une responsabilité s'est retrouvée dans la mauvaise couche.

Questions à se poser

- Que se passerait-il si `Joueur` avait besoin d'importer `Inscription` ? Est-ce que le programme compilerait encore ? Est-ce que ce serait souhaitable ?
- Les champs de `Partie` étaient accessibles depuis `Inscription` tant que tout était dans le même package. Le sont-ils encore ?

Un dernier point de vocabulaire : ce découpage en couches est une forme de séparation des responsabilités, la même idée que `CC-CLS.1` applique à une classe. Ce qui vaut pour une classe vaut pour un package.

PKG.5 — `protected`, ou ce que l'héritage ouvre

Package : ***`be.ephec.java.pkg.e05`***

Objectif

Comprendre la seule visibilité dont la portée ne se décrit pas par un périmètre.

Pourquoi cet exercice arrive maintenant

`PKG.1` à `PKG.4` ont laissé une case vide : `protected` y était mentionné sans être utilisé, parce qu'il ne prend son sens qu'avec l'héritage. Vous l'avez maintenant.

Visibilité	Qui peut atteindre
`private`	`la classe elle-même, et rien d'autre`
`(rien)`	`le même package`
`protected`	`le même package, ET les sous-classes où qu'elles soient`

Visibilité	Qui peut atteindre
`public`	`tout le monde`

Ce qui vous est fourni

La classe `Capteur`, dans `pkg.e05.base`. Elle présente trois champs avec trois visibilités différentes.

```
private final String numeroDeSerie;      personne d'autre
protected double dernieremesure;         les sous-classes
public final String nom;                 tout le monde
```

Énoncé

Dans un **autre package** — `pkg.e05.derive` — complétez deux classes qui n'ont pas le même rapport à Capteur.

Classe	Son rapport à Capteur	Ce qu'elle doit faire
`CapteurMoyenne`	`elle en HÉRITE`	lisser une mesure, et se décrire
`Tableau`	`elle s'en sert seulement`	additionner des mesures, trouver la plus élevée

L'expérience à mener

C'est elle, le cœur de l'exercice. Dans chacune des deux classes, essayez d'écrire les trois accès, et notez le message exact du compilateur quand il refuse.

```
                       depuis CapteurMoyenne    depuis Tableau
  nom                       ?                        ?
  dernieremesure            ?                        ?
  numeroDeSerie             ?                        ?
```

Écrivez ensuite en une phrase ce que `protected` ouvre exactement. Les deux classes sont dans le même package : si le résultat diffère, ce n'est pas le package qui décide.

Deux contraintes

- Dans `lisser()`, accédez **directement** au champ `protected`, sans passer par un accesseur. C'est ce qu'on cherche à rendre visible.
- Dans `decrire()`, le numéro de série vous est inaccessible. Une méthode publique de la classe mère vous en donne une forme utilisable — trouvez-la.

Comment vérifier votre code

```
lisser(60) sur un capteur neuf             30.00
puis lisser(70)                            50.00
decrire()                                  Hygrometre (9b71) : 50.00
sommeDesMesures des deux capteurs          71.50
plusEleve()                                Hygrometre
plusEleve() sans capteur                   aucun capteur
```

Questions à se poser

- Les deux classes sont dans le même package et n'ont pas les mêmes droits. Qu'est-ce qui les distingue ?
- Dans `lisser()`, vous auriez pu écrire `mesurer((valeur + lire()) / 2)` et vous passer de `protected`. Laquelle des deux versions préférez-vous, et pourquoi ? Voir `CC-OBJ.2`.
- Une sous-classe peut-elle atteindre `dernieremesure` sur *un autre* capteur qu'elle-même ? Essayez.

Module COL — Collections

Les exercices du module. Leurs codes sont **cliquables** : ils mènent directement à l'exercice.

Exercice	Package	Sujet
COL.1	`be.ephec.java.col.e01`	Deux façons d'être trié
COL.2	`be.ephec.java.col.e02`	Un annuaire
COL.3	`be.ephec.java.col.e03`	Lire des noms et les trier
COL.4	`be.ephec.java.col.e04`	Compter les mots d'un texte
COL.5	`be.ephec.java.col.e05`	Filtrer avec un prédicat
COL.6	`be.ephec.java.col.e06`	Un ensemble sans doublons
COL.7	`be.ephec.java.col.e07`	Une table de collections
COL.8	`be.ephec.java.col.e08`	Trois ordres pour une bibliothèque

Trois familles, trois promesses

Un tableau a une taille fixe et ne sait rien faire d'autre que ranger. Les collections grandissent toutes seules et portent chacune une promesse différente.

Famille	Ce qu'elle garantit	Représentants
`List`	un ordre, des doublons possibles, un accès par position	`ArrayList, LinkedList`
`Set`	aucun doublon	`HashSet, TreeSet`
`Map`	une valeur par clé	`HashMap, TreeMap`

Le choix se fait sur la promesse, pas sur l'habitude. Si les doublons n'ont pas de sens, un `Set` l'interdit — et vous n'aurez plus à y penser. Si l'ordre compte, un `TreeSet` ou une `TreeMap` veillent à maintenir l'ordre en permanence. Confier une règle à la structure vaut mieux que de compter sur la vigilance de celui qui l'utilise.

Vous verrez au fil des exercices que ce choix se lit dans la déclaration : `Map<String, Set<String>>` dit à lui seul qu'un étudiant a des cours, sans répétition possible.

COL.1 — Deux façons d'être trié

Package : ***`be.ephec.java.col.e01`***

Objectif

Distinguer une collection qu'on trie d'une collection qui reste triée.

Énoncé

Complétez la classe `Tri`, qui expose deux méthodes statiques.

Élément	Signature	Rôle
`triee`	`public static List<Integer> triee(List<Integer>)`	Une NOUVELLE liste triée ; la liste reçue est intacte
`ensembleTrie`	`public static Set<Integer> ensembleTrie(List<Integer>)`	Un `TreeSet`, trié en permanence

Indice : la classe `Collections` et sa méthode statique `sort`. Attention, elle trie sur place et ne retourne rien — d'où la copie à faire avant.

Écrivez ensuite le `Programme` : il demande n, tire n entiers au hasard entre 0 et 1000, et affiche les deux résultats.

Comment vérifier votre code

> `be.ephec.java.col.e01.TriTest`
>
> Vérifie le tri, le fait que la liste reçue ne soit pas modifiée, le cas de la liste vide, et que le `TreeSet` soit à la fois trié et débarrassé des doublons.

Questions à se poser

- Comparez la taille de la liste et celle de l'ensemble, sur 100 tirages entre 0 et 1000. Pourquoi diffèrent-elles ?
- Un `TreeSet` est trié en permanence. Qu'est-ce que cela coûte, et quand ce coût en vaut-il la peine ?
- Que se passerait-il si `triee` triait la liste reçue au lieu d'en copier une ? Voir `CC-MET.7` du mémento Clean Code.

COL.2 — Un annuaire

Package : `be.ephec.java.col.e02`

Objectif

Se servir d'une table associative, et découvrir que le choix de son implémentation change ce qu'on peut en attendre.

Énoncé

Le record `Fiche` vous est fourni : un nom, un numéro, une adresse. Complétez la classe `Annuaire`, qui associe un nom à une fiche.

Élément	Signature	Rôle
`ajouter`	`public void ajouter(Fiche)`	Indexe la fiche par son nom
`chercher`	`public Fiche chercher(String)`	La fiche, ou `null` si le nom est inconnu
`taille`	`public int taille()`	Le nombre de fiches
`nomsDansLOrdre`	`public List<String> nomsDansLOrdre()`	Les noms, par ordre alphabétique

Commencez par une `HashMap`, comme le demande l'énoncé d'origine.

> **La question qui fait l'exercice**
>
> Peut-on faire en sorte que les fiches apparaissent dans l'ordre des noms ?
>
> Répondez d'abord avec la `HashMap`, en triant les clés au moment de les retourner. Puis remplacez la `HashMap` par une `TreeMap` et regardez ce que devient `nomsDansLOrdre` : une seule ligne change dans toute la classe, et le test passe toujours.
>
> Lequel des deux préférez-vous, et pourquoi ? La réponse dépend de la fréquence à laquelle on demande cet ordre.

Comment vérifier votre code

> `be.ephec.java.col.e02.AnnuaireTest`
>
> Vérifie la recherche, le cas d'un nom absent, le remplacement d'une fiche de même nom, l'ordre alphabétique et le cas de l'annuaire vide.

Questions à se poser

- Ajouter deux fiches de même nom ne fait pas grandir l'annuaire. Où est-ce écrit dans votre code ? Regardez ce que retourne `put` dans la Javadoc.
- chercher retourne `null` pour un nom inconnu. L'énoncé le demande, mais est-ce une bonne idée ? Voir `CC-ERR.2` — et comparez avec ce que fait `COL.7`.

COL.3 — Lire des noms et les trier

Package : ***`be.ephec.java.col.e03`***

Objectif

Remplir une collection à partir d'un flux de texte, puis la trier.

Contexte

Dans un vrai programme, les noms viendraient d'un fichier. Trois lignes suffisent :

```
File fichier = new File("etudiants.txt");
Scanner scanner = new Scanner(fichier);
```

La classe fournie `SourceDeTexte` vous retourne un `Scanner` déjà ouvert, pour que l'exercice porte sur les collections et non sur les chemins de fichiers — sujet du module `FIC`. Le reste de votre code serait identique : un `Scanner` se lit de la même façon, quelle que soit sa source.

Énoncé

Élément	Signature	Rôle
`lire`	`public static List<String> lire(Scanner)`	Tous les noms, un par ligne, dans l'ordre de lecture
`triee`	`public static List<String> triee(List<String>)`	Une nouvelle liste, par ordre alphabétique

Indice : `scanner.hasNextLine()` et `scanner.nextLine()`.

Comment vérifier votre code

> `be.ephec.java.col.e03.NomsTest`
>
> Vérifie la lecture, le cas d'une source vide, le tri sans modification de la liste reçue, et le contenu de la source fournie.
>
> Remarquez que le test passe au `Scanner` une simple chaîne de caractères. C'est possible parce que lire ne sait pas d'où vient son `Scanner` — et c'est ce qui la rend testable sans dépendre d'aucun fichier.

Questions à se poser

- `nextLine()` lit une ligne, `next()` lit un mot. Que se passerait-il si un nom contenait une espace et que vous utilisiez `next()` ?
- Le tri est alphabétique. Où est définie cette notion d'ordre pour des `String`, sachant que vous n'avez rien écrit à ce sujet ?

COL.4 — Compter les mots d'un texte

Package : `be.ephec.java.col.e04`

Objectif

L'usage le plus courant d'une table associative : accumuler un décompte.

Énoncé

Élément	Signature	Rôle
`compter`	`public static Map<String, Integer> compter(Scanner)`	Combien de fois chaque mot apparaît
`rapport`	`public static String rapport(Map<String, Integer>)`	Le décompte en texte, trié

Indice pour compter : cherchez `getOrDefault` dans la Javadoc de `Map`. Elle remplace « si le mot est absent mettre 1, sinon incrémenter » par une seule ligne.

Le format du rapport est imposé, un mot par ligne, par ordre alphabétique, sans retour à la ligne final :

```
abeille : 1
chat : 1
zebre : 2
```

La question de l'ordre

Dans le `Programme`, affichez la `Map` directement, sans passer par `rapport`. Dans quel ordre les mots sortent-ils ? Recommencez avec une `TreeMap` à la place de la `HashMap` et comparez.

Retenez ce que vous observez : une `HashMap` ne garantit aucun ordre. Celui que vous voyez dépend du code de hachage des clés, il paraît souvent aléatoire, et surtout il n'est pas stable d'une version de Java à l'autre. Un programme qui s'appuierait dessus fonctionnerait aujourd'hui et pas demain.

Comment vérifier votre code

> `be.ephec.java.col.e04.OccurrencesTest`
>
> Vérifie le décompte, le cas d'un mot absent, celui d'un texte vide, le format exact du rapport, et le texte fourni.

Questions à se poser

- Le rapport trie les clés au lieu de supposer un ordre. Que serait devenu le test si vous aviez parcouru la `HashMap` telle quelle ?
- « Le » et « le » comptent-ils pour le même mot dans votre version ? Est-ce souhaitable, et que faudrait-il changer ?

COL.5 — Filtrer avec un prédicat

Package : `be.ephec.java.col.e05`

Objectif

Passer un comportement en paramètre à une méthode d'une collection.

Contexte

L'interface `Collection` déclare :

```
boolean removeIf(Predicate<? super E> filtre)
```

Elle supprime tous les éléments pour lesquels le filtre répond `true`. Et `Predicate` est une interface du JDK : elle déclare une seule méthode, `boolean test(T valeur)`. Vous savez donc déjà tout faire — c'est exactement `ITF.1`, avec une interface écrite par quelqu'un d'autre.

Énoncé

Complétez la classe `EstPair`, qui implémente `Predicate<Integer>`, puis les deux méthodes de `Filtrage`.

Élément	Signature	Rôle
`test`	`public boolean test(Integer)`	Répond si le nombre est pair
`supprimerLesPairs`	`public static void supprimerLesPairs(List<Integer>)`	Modifie la liste reçue
`compterLesPairs`	`public static int compterLesPairs(List<Integer>)`	Compte sans rien supprimer

> **Pourquoi une classe nommée**
>
> Écrire une classe entière pour une seule expression est verbeux, et il existe deux écritures plus courtes que vous verrez au module `LAM` : la classe anonyme, puis la lambda. Les trois font exactement la même chose.
>
> La classe nommée a pourtant un avantage qu'on oublie : elle porte un nom, elle se réutilise, et elle se teste séparément — ce que fait justement la classe de test.

Comment vérifier votre code

> `be.ephec.java.col.e05.FiltrageTest`
>
> Vérifie le prédicat, y compris sur zéro et sur les négatifs, le comptage qui ne supprime rien, la suppression, et le fonctionnement sur une `LinkedList`.

Questions à se poser

- Essayez de supprimer les pairs avec une boucle `for` et un `remove` au passage. Quelle exception obtenez-vous, et pourquoi `removeIf` existe-t-elle ?
- `removeIf` est déclarée sur `Collection`, pas sur `List`. Sur quelles autres structures fonctionne-t-elle donc ?

COL.6 — Un ensemble sans doublons

Package : `be.ephec.java.col.e06`

Objectif

Constater les deux services que rend un `Set` : l'unicité, et la rapidité de la recherche.

Énoncé

Élément	Signature	Rôle
`sansDoublons`	`public static Set<Integer> sansDoublons(List<Integer>)`	Les mêmes valeurs, une seule fois chacune
`aEteTire`	`public static boolean aEteTire(Set<Integer>, int)`	Le nombre figure-t-il dans l'ensemble
`doublons`	`public static int doublons(List<Integer>)`	Combien de valeurs ont été perdues

Indice : `HashSet` a un constructeur qui accepte n'importe quelle `Collection`. Et pour compter les doublons, une soustraction suffit — inutile de compter les répétitions une par une.

Comment vérifier votre code

> `be.ephec.java.col.e06.TirageTest`
> Vérifie l'élimination des doublons, l'appartenance, le décompte des doublons perdus, et le cas de la liste vide.

Questions à se poser

- Un `Set` répond à « me contiens-tu ? » sans parcourir ses éléments. Comment est-ce possible, et qu'est-ce qui rend cela impossible pour une `List` ?
- Sur 100 tirages entre 1 et 1000, combien de doublons obtenez-vous en moyenne ? Le chiffre est plus élevé que l'intuition ne le suggère.
- Que perdez-vous en passant d'une `List` à un `Set`, en dehors des doublons ?

COL.7 — Une table de collections

Package : ***`be.ephec.java.col.e07`***

Objectif

Emboîter deux structures, et laisser la structure porter une règle métier.

Énoncé

Complétez la classe `Inscriptions`, qui associe à chaque étudiant l'ensemble des cours qu'il suit — une `Map<String, Set<String>>`.

Élément	Signature	Rôle
`inscrire`	`public void inscrire(String, String)`	Inscrit un étudiant à un cours
`coursDe`	`public Set<String> coursDe(String)`	Ses cours, ou un ensemble VIDE s'il est inconnu
`etudiants`	`public List<String> etudiants()`	Les noms, par ordre alphabétique
`rapport`	`public String rapport()`	Un étudiant par ligne, cours triés

```
Lovelace : Algebre, Java
Turing : Java
```

Indice : cherchez `computeIfAbsent` dans la Javadoc de `Map`. Elle remplace « si l'étudiant n'existe pas encore, lui créer un ensemble vide, puis y ajouter le cours ».

Deux décisions de conception

Un `Set` et non une `List` pour les cours. Un étudiant ne s'inscrit pas deux fois au même cours. Écrit ainsi, c'est la structure qui l'interdit — personne n'a besoin d'y penser au moment d'appeler `inscrire`.

Un ensemble vide plutôt que `null`. `coursDe` d'un étudiant inconnu retourne un ensemble vide, sur lequel l'appelant peut boucler sans rien vérifier. Comparez avec le `chercher` de `COL.2`, qui retourne `null` — et voyez lequel des deux vous préféreriez utiliser. C'est la règle `CC-ERR.2` du mémento Clean Code.

Comment vérifier votre code

> `be.ephec.java.col.e07.InscriptionsTest`
>
> Vérifie l'inscription à plusieurs cours, l'ensemble vide pour un inconnu, l'absence d'effet d'une inscription en double, l'ordre alphabétique et le format du rapport.

Questions à se poser

- Écrivez `inscrire` sans `computeIfAbsent`, avec un test explicite. Combien de lignes, et laquelle des deux versions préférez-vous relire dans six mois ?
- Quelle structure choisiriez-vous pour répondre à la question inverse : quels étudiants suivent le cours de Java ?

COL.8 — Trois ordres pour une bibliothèque

Package : ***`be.ephec.java.col.e08`***

Objectif

Donner un ordre naturel à une classe avec `Comparable`, puis lui fournir d'autres ordres de l'extérieur avec `Comparator`.

Énoncé

Le record `Livre` a un titre, un auteur et une année de parution. Complétez les quatre classes suivantes ; le `Programme` est fourni, et affiche la même bibliothèque dans les trois ordres.

Élément	Signature	Rôle
`Livre`	`public int compareTo(Livre autre)`	L'ordre naturel : par titre, alphabétiquement
`ParAnnee`	`public int compare(Livre premier, Livre second)`	Par année, puis par titre à égalité
`ParAuteur`	`public int compare(Livre premier, Livre second)`	Par auteur, puis par titre
`parTitre`	`public static List<Livre> parTitre(List<Livre>)`	Une nouvelle liste, dans l'ordre naturel
`parAnnee`	`public static List<Livre> parAnnee(List<Livre>)`	Une nouvelle liste, triée avec `ParAnnee`
`rangesParAuteur`	`public static Set<Livre> rangesParAuteur(List<Livre>)`	Un `TreeSet` construit avec `ParAuteur`

Indices : une `String` sait déjà se comparer ; pour les années, `Integer.compare` plutôt qu'une soustraction (voir `COL §6`). La méthode `sort` d'une `List` reçoit un comparateur, et le constructeur de `TreeSet` aussi. Comme dans `COL.1`, aucune méthode de `Tris` ne modifie la liste reçue.

Comment vérifier votre code

> `be.ephec.java.col.e08.TrisTest`
>
> Vérifie l'ordre naturel, les trois tris, le départage à égalité, que l'ensemble garde tous les livres, et que la liste reçue n'est pas modifiée.

Questions à se poser

- Retirez le second critère de `ParAuteur` (le titre), et relancez le test. Combien de livres reste-t-il dans l'ensemble, et pourquoi ?

- Pourquoi l'ordre par titre est-il l'ordre naturel, et pas l'ordre par année ? Qu'est-ce qui rend un ordre « naturel » pour une classe ?
- Au module `LAM`, chacun des deux comparateurs tiendra en une ligne. Lesquelles de ces classes vous paraîtront alors encore utiles ?

Module EXC — Exceptions

Les exercices du module. Leurs codes sont **cliquables** : ils mènent directement à l'exercice.

Exercice	Package	Sujet
EXC.1	`be.ephec.java.exc.e01`	Lever sa propre exception
EXC.2	`be.ephec.java.exc.e02`	Un compte, sans exception
EXC.3	`be.ephec.java.exc.e03`	Un numéro de compte vérifié
EXC.4	`be.ephec.java.exc.e04`	Provoquer, puis traiter
EXC.5	`be.ephec.java.exc.e05`	Lire une trace

Ce qu'il faut avoir en tête

Une exception est un objet. La lever interrompt le cours normal du programme et remonte la pile des appels jusqu'à trouver quelqu'un qui l'attrape. Si personne ne l'attrape, le programme s'arrête et affiche une trace.

```
throw new MonException("ce qui ne va pas");     // lever

try {
    quelqueChoseQuiPeutEchouer();
} catch (MonException e) {
    System.out.println(e.getMessage());          // attraper
}
```

Contrôlée ou non contrôlée

	Contrôlée	Non contrôlée
Hérite de	`Exception`	`RuntimeException`
Le compilateur	oblige à traiter ou à déclarer	n'exige rien
À utiliser pour	une erreur prévisible dont l'appelant doit se soucier	une erreur de programmation
Exemples	fichier introuvable, IBAN invalide	division par zéro, indice hors limites

Vos exceptions à vous seront contrôlées dans `EXC.1` et `EXC.3`. Celles d'`EXC.4` sont levées par la JVM et ne sont pas contrôlées : le code compile sans `try`, et plante à l'exécution. C'est tout le piège.

EXC.1 — Lever sa propre exception

Package : ***`be.ephec.java.exc.e01`***

Objectif

Écrire une exception, la lever au bon moment, et laisser l'objet dans un état correct quand on la lève.

Énoncé

Complétez `MoveException` — une classe qui hérite d'`Exception` et transmet un message à sa classe mère — puis la méthode `deplace(int dx, int dy)` de `Point`.

Un point ne peut pas sortir du quadrant positif. Si l'une des coordonnées deviendrait négative, le déplacement n'a pas lieu et la méthode lève l'exception, avec ce message exact :

```
Deplacement refuse : (-3, 5) sort du quadrant positif
```

Le vrai piège de l'exercice

Le test vérifie qu'après un déplacement refusé, le point n'a pas bougé du tout.

Si vous écrivez dans les champs avant de vérifier, l'objet reste à moitié déplacé chaque fois que l'exception est levée. Une méthode qui échoue doit laisser l'objet tel qu'elle l'a trouvé — calculez d'abord, vérifiez, et n'écrivez qu'après.

Comment vérifier votre code

`be.ephec.java.exc.e01.PointTest`

Vérifie que l'exception est contrôlée, le déplacement valide, le refus, l'intégrité du point après refus, le message exact, et que zéro reste accepté.

Questions à se poser

- Faites hériter `MoveException` de `RuntimeException` au lieu d'`Exception`. Que devient le `try` de votre programme ? Est-ce un progrès ?
- Un déplacement refusé est-il une erreur de programmation ou un événement normal ? Votre réponse justifie le choix de la classe mère.

EXC.2 — Un compte, sans exception

Package : *`be.ephec.java.exc.e02`*

Objectif

Écrire d'abord la version sans exception, pour voir précisément ce qui lui manque.

Énoncé

Complétez la classe `Compte`. Un compte qui vient d'être créé est à zéro.

Élément	Signature	Rôle
`getSolde`	`public double getSolde()`	Le solde courant
`ajoute`	`public void ajoute(double)`	Ajoute au solde
`retire`	`public double retire(double)`	Retire, et retourne ce qui a réellement été retiré

On ne peut pas retirer plus que le solde : sur un compte à 100, un retrait de 120 retourne 100 et laisse le solde à zéro.

Écrivez ensuite `GestionDeCompte` et faites les trois essais : un dépôt, un retrait possible, un retrait excessif.

La question à garder

Sur un compte à 100, un retrait de 100 et un retrait de 120 retournent tous deux 100. L'appelant peut-il savoir si son retrait a été complet ? Que devrait-il faire pour s'en assurer ?

Gardez votre réponse : c'est exactement le problème que l'exercice suivant résout.

Comment vérifier votre code

`be.ephec.java.exc.e02.CompteTest`

Vérifie un nouveau compte, le dépôt, le retrait possible, le retrait plafonné, et le retrait sur un compte vide.

EXC.3 — Un numéro de compte vérifié

Package : `be.ephec.java.exc.e03`

Objectif

Valider dans le constructeur, avec une famille d'exceptions plutôt qu'une seule — ou l'inverse, selon ce dont l'appelant a besoin.

Contexte : le contrôle d'un IBAN

Un compte belge porte un numéro de la forme BECC NNNN NNNN NNNN. Sa validité se vérifie ainsi :

```
1. retirer tout ce qui n'est ni lettre ni chiffre
2. deplacer les quatre premiers caracteres a la fin
3. passer en majuscules
4. remplacer chaque lettre par sa position dans l'alphabet plus 9
     A vaut 10, B vaut 11, ... Z vaut 35
5. le reste de la division du nombre obtenu par 97 doit valoir 1
```

Le nombre obtenu dépasse largement un `long`. Utilisez `java.math.BigInteger`, qui n'a pas de limite de taille.

Énoncé

Complétez les quatre classes d'exception, puis la classe `Compte`. Le constructeur reçoit le numéro et le vérifie en trois temps, dans cet ordre : la longueur, le préfixe, le contrôle modulo 97.

Erreur	`Exception`	Message imposé
longueur	`IbanLongueurException`	L'IBAN doit contenir 16 caracteres, en contient 12
préfixe	`IbanPrefixeException`	L'IBAN doit commencer par BE, commence par FR
modulo	`IbanModuloException`	Le controle modulo 97 est incorrect : 28 au lieu de 1

Les trois héritent d'`IbanInvalideException`, qui hérite d'`Exception`. Le numéro est mémorisé sous sa forme nettoyée, sans espaces.

Les messages sont vérifiés au caractère près. Chacun dit ce qui était attendu ET ce qui a été reçu : « IBAN invalide » n'aiderait personne à corriger sa saisie. Voir `CC-ERR.4` du mémento Clean Code.

La question qui fait l'exercice

> **Trois exceptions ou une seule ?**
>
> Dans `GestionDeCompte`, essayez les quatre numéros donnés en commentaire. Combien de blocs `catch` écrivez-vous ?
>
> Si votre réaction est la même dans les trois cas — afficher le problème — un seul `catch` sur la classe mère suffit, et trois blocs identiques n'apporteraient rien. Si elle devait différer — proposer une correction pour une faute de frappe, rediriger vers une banque étrangère — alors les trois classes distinctes prennent tout leur sens.
>
> C'est la règle `CC-ERR.5` : on ne définit pas une exception par cause technique, mais selon ce que l'appelant doit distinguer. Ici, la hiérarchie offre les deux.

Comment vérifier votre code

> `be.ephec.java.exc.e03.CompteTest`
>
> Vérifie la hiérarchie des exceptions, le calcul du modulo, l'acceptation d'un IBAN valide, les trois refus avec leur message exact, l'ordre des contrôles, et qu'un seul `catch` attrape bien les trois.

Questions à se poser

- La validation est dans le constructeur. Qu'est-ce que cela garantit qu'une méthode `valider()` appelée séparément ne garantirait pas ?
- Écrivez deux `catch`, d'abord sur `IbanInvalideException` puis sur `IbanPrefixeException`. Que dit le compilateur, et pourquoi a-t-il raison ?

EXC.4 — Provoquer, puis traiter

Package : `be.ephec.java.exc.e04`

Objectif

Rencontrer des exceptions qu'on n'a pas écrites, et que le compilateur ne signale pas.

Énoncé — première partie

Écrivez d'abord le Programme SANS aucun `try`. Il demande la taille d'un tableau, le crée, demande une position, et y place la valeur 12.

```
int[] tableau = new int[taille];
tableau[position] = 12;
```

Lancez-le deux fois : une taille négative, puis une position hors limites. Notez les deux messages d'erreur — c'est en les lisant que vous saurez quelles exceptions attraper. Remarquez surtout que le code a compilé sans rien dire.

Énoncé — seconde partie

Complétez `Tableau.placerDouze(int taille, int position)`, qui fait la même chose mais ne laisse jamais échapper d'exception. Elle retourne un compte-rendu, aux formulations imposées :

```
placerDouze(5, 2)    ->  "12 place en position 2 d'un tableau de 5 cases"
placerDouze(-3, 0)   ->  "Taille impossible : -3"
placerDouze(5, 7)    ->  "Position 7 hors du tableau de 5 cases"
```

Le programme, et pourquoi il boucle

Annoncez d'abord ce que le programme fait. Sans cela l'utilisateur voit surgir deux questions sans savoir ce qu'on attend de lui. Dites aussi ce qui peut mal tourner : c'est le sujet de l'exercice, pas un accident.

Demandez ensuite la taille **une seule fois**, puis **bouclez sur la position** jusqu'à ce que l'utilisateur entre `-6`. Rappelez-lui les positions valides et la façon de sortir.

La boucle n'est pas un ornement. Elle permet d'essayer plusieurs positions dans le même tableau, et c'est en les essayant qu'on voit où sont les limites. Redémarrer le programme entre chaque essai décourage l'expérimentation — or c'est elle qui enseigne ici.

> **Un message calculé peut devenir absurde**
>
> « Positions valides : 0 à taille - 1 » donne « 0 à -4 » si l'utilisateur entre -3. Le message est juste au sens du calcul, et il ne veut rien dire.
>
> Prévoyez le cas. C'est une habitude à prendre : dès qu'un texte affiché contient une valeur calculée, demandez-vous quelle saisie le rendrait ridicule.

Vérifiez enfin ce que la boucle change. Avec votre première version, sans `try`, le programme s'arrête net à la première erreur : la boucle ne repart pas. Avec les `try`, elle survit et l'utilisateur peut réessayer. C'est exactement ce que l'exercice veut montrer.

Comment vérifier votre code

> `be.ephec.java.exc.e04.TableauTest`
>
> Vérifie le cas normal, les deux bornes valides, la taille négative, la position hors limites dans les deux sens, et surtout qu'aucune exception ne sort de la méthode.

Questions à se poser

- Un indice négatif et un indice trop grand lèvent la même exception. Cela vous paraît-il logique ?
- Faut-il mettre tout le corps de la méthode dans le `try`, ou seulement ce qui peut échouer ? Que gagne-t-on à réduire le bloc ?
- Ces exceptions ne sont pas contrôlées : le compilateur ne vous oblige à rien. Pourquoi ce choix, à votre avis, alors qu'un IBAN invalide l'est ?

EXC.5 — Lire une trace

Package : `be.ephec.java.exc.e05`

Objectif

Savoir lire ce que le programme affiche quand il s'arrête — et comprendre que le lieu de l'erreur n'est pas toujours sa cause.

Énoncé

Le package contient un petit jeu de rôle en quatre classes. Lancez `ProgrammeJdR`. Il plante. **Lisez la trace avant de toucher au code.**

Complétez ensuite la classe Diagnostic, dont chaque méthode répond à une question sur ce que vous avez lu.

Élément	Ce qu'il faut répondre
`typeDException`	Le nom simple de l'exception levée
`messageDeLException`	Ce qui suit les deux points, sur la première ligne
`classeFautive`	La classe où l'exception a été levée
`methodeFautive`	Le nom de la méthode, tel qu'il figure dans la trace
`methodeAppelante`	Celle qui a appelé le code fautif

Attention à `methodeFautive` : ce n'est pas forcément un nom que vous avez écrit. Recopiez exactement ce que dit la trace, ponctuation comprise. La réponse vous apprendra quelque chose sur la façon dont la JVM désigne certaines méthodes.

Comment se lit une trace

La première ligne donne le type de l'exception et son message. Les lignes « at » qui suivent remontent la chaîne des appels : la première dit où l'erreur s'est produite, la dernière est toujours le point d'entrée du programme.

C'est la pile d'appels décrite en `TEC-EXE.1` du mémento technique, affichée au moment où tout s'arrête. Le numéro de ligne entre parenthèses est cliquable dans IntelliJ.

La correction

Complétez enfin `degatsParCibleSansPlanter`, qui retourne les dégâts par cible, ou 0 si le sort ne vise personne. **Vous ne pouvez pas modifier la classe fautive :** traitez le problème là où vous avez la main.

Comment vérifier votre code

> `be.ephec.java.exc.e05.DiagnosticTest`
>
> Vérifie vos cinq réponses et la correction. Les messages d'échec vous mettent sur la voie si vous vous trompez.

Questions à se poser

- L'erreur est dans une classe, mais la cause est ailleurs : quelle méthode a mis le programme dans cette situation ? Une trace sert autant à trouver le coupable qu'à remonter jusqu'à celui qui l'a mis en position de faillir.
- On aurait pu entourer l'appel d'un `try/catch` au lieu de tester le nombre de cibles. Les deux fonctionnent — lequel préférez-vous, pour un cas parfaitement prévisible ?

Module JUN — Tests unitaires avec JUnit

Les exercices du module. Leurs codes sont **cliquables** : ils mènent directement à l'exercice.

Exercice	Package	Sujet
JUN.1	be.ephec.java.jun.e01	Le test comme spécification
JUN.2	be.ephec.java.jun.e02	Détecter un défaut
JUN.3	be.ephec.java.jun.e03	Écrire une classe de test

Ce module fonctionne à l'envers

Depuis le début de l'année, les classes de test vous sont fournies et vous écrivez le code qui les fait passer. Ici, c'est l'inverse : c'est vous qui devez écrire les classes de test pour du code qu'on vous fournit.

D'où une difficulté que ce module ne cache pas. Vérifier automatiquement du code est facile — c'est ce que fait chaque exercice depuis janvier. Vérifier automatiquement un test l'est beaucoup moins, parce qu'un test qui ne vérifie rien passe au vert aussi bien qu'un bon.

C'est la raison d'être des trois exercices. `JUN.1` vous fait lire un test comme une spécification. `JUN.2` vous confronte à du code volontairement défectueux, ce qui rend la qualité de votre vérification mesurable. `JUN.3` vous fait écrire une vraie classe de test.

> **Le défaut le plus fréquent, et le plus difficile à voir**
>
> Une méthode annotée `@Test` qui se contente d'appeler du code passe au vert quoi qu'il arrive : elle n'échoue que si une exception inattendue survient. Elle donne une impression de couverture sans vérifier aucun comportement.
>
> C'est l'erreur la plus répandue dans les classes de test rendues à l'examen. Avant de considérer un test comme écrit, cherchez son assertion. Voir `CC-TST.2` du mémento Clean Code.

Les trois temps d'un test

```
@Test
@DisplayName("Un retrait diminue le contenu du montant retire")
void unRetraitDiminueLeContenu() throws ContenuInsuffisant {
    Tirelire tirelire = new TirelireCorrecte();     // preparer
    tirelire.depot(50);

    tirelire.retrait(20);                            // agir

    assertEquals(30, tirelire.getContenu(), 0.001);  // verifier
}
```

Préparer, agir, vérifier - séparés par des lignes vides. C'est la structure à suivre pour vos propres tests. Vous ne la retrouverez pas dans tous les tests fournis : beaucoup vérifient seulement la forme d'une classe (est-elle abstraite ? ce champ existe-t-il ?), en une ligne, sans rien à préparer ni à exécuter.

JUN.1 — Le test comme spécification

Package : *`be.ephec.java.jun.e01`*

Objectif

Découvrir qu'un bon test se lit avant d'être exécuté.

Énoncé

Ouvrez `CalculatriceTest` avant d'écrire une ligne. Il est fourni complet, et il constitue tout l'énoncé : vous y trouverez les signatures, les valeurs attendues, et ce qui doit se produire quand on divise par zéro.

Complétez ensuite la classe `Calculatrice` pour que les treize tests passent au vert.

C'est le principe du développement piloté par les tests : on écrit le test d'abord, on le voit échouer, puis on écrit le code qui le fait passer. Vous n'aurez à faire que les deux dernières étapes.

Deux choses ne s'inventent pas

Elles ne figurent que dans le test, et nulle part ailleurs. **La division entière tronque** — 11 / 4 vaut 2, pas 3. Et **la division par zéro doit laisser passer l'exception** : ne retournez pas 0, n'attrapez rien.

Ce second point mérite qu'on s'y arrête. Retourner 0 ferait passer une erreur pour un résultat, et le programme continuerait avec une valeur fausse. C'est exactement ce que `CC-ERR.1` du mémento Clean Code proscrit.

Comment vérifier votre code

> `be.ephec.java.jun.e01.CalculatriceTest`
>
> Treize tests : trois par opération, chacun sur une seule valeur, et un pour la division par zéro. Ils sont votre énoncé autant que votre vérification.

Questions à se poser

- Lisez les `@DisplayName`. Sauriez-vous écrire la classe uniquement à partir d'eux, sans lire le corps des tests ? C'est le but qu'ils poursuivent.
- Un test contient plusieurs `assertEquals`. Cela contredit-il `CC-TST.2`, qui demande de ne vérifier qu'une chose par test ?
- Le message passé au troisième argument d'un `assertEquals` ne s'affiche qu'en cas d'échec. À quoi sert-il, et quand vaut-il la peine d'en écrire un ?

JUN.2 — Détecter un défaut

Package : *`be.ephec.java.jun.e02`*

Objectif

Mesurer la qualité d'une vérification à ce qu'elle est capable de démasquer.

Contexte

L'interface `Tirelire` définit un contrat en trois méthodes : `depot`, `retrait` — qui lève `ContenuInsuffisant` si le montant dépasse le contenu — et `getContenu`.

Quatre classes l'implémentent. Une seule respecte le contrat ; les trois autres classes ont chacune un défaut différent.

Énoncé

Complétez `Verificateur.estCorrecte(Tirelire)`, qui retourne `true` si la tirelire respecte son contrat et `false` sinon. Elle doit démasquer les trois implémentations défectueuses et laisser passer la bonne.

Le contrat compte cinq clauses. Vérifiez-les une par une :

- une tirelire neuve est vide ;
- un dépôt augmente le contenu d'autant ;
- un retrait possible diminue le contenu d'autant ;

- un retrait trop grand lève `ContenuInsuffisant` ;
- et dans ce cas, le contenu n'a pas bougé.

> **Ce n'est pas encore une classe de test `JUnit`**
>
> C'est le raisonnement qu'une classe de test met en pratique, écrit à la main pour qu'on puisse le vérifier automatiquement. Chaque clause vérifiée correspond à ce que serait une méthode `@Test`.
>
> Ne lisez pas le code des implémentations pour trouver leurs défauts. Repérer un bug en lisant ne prouve rien ; un test qui échoue, si — et c'est ce qui continuera de fonctionner quand le code changera dans six mois.

Comment vérifier votre code

> `be.ephec.java.jun.e02.VerificateurTest`
>
> Confronte votre vérificateur à la bonne implémentation et aux trois classes défectueuses. Chaque message d'échec vous indique ce que la vérification manquante aurait dû observer.
>
> Un dernier test refuse un vérificateur qui retournerait toujours la même réponse : c'est exactement ce que fait une classe de test sans assertion.

Questions à se poser

- Une seule des trois implémentations défectueuses de `Tirelire` lève correctement l'exception, et se trompe ailleurs. Laquelle, et quelle clause l'a démasquée ?
- Si vous aviez oublié la cinquième clause — le contenu inchangé après un refus — laquelle des trois serait passée à travers ?
- Comparez avec `EXC.1`, où le test vérifiait qu'un point ne bouge pas quand le déplacement est refusé. La même idée : vérifier l'état après l'échec, pas seulement l'échec.

JUN.3 — Écrire une classe de test

Package : ***`be.ephec.java.jun.e03`***

Objectif

Écrire, cette fois, une vraie classe de test `JUnit`.

Énoncé

La classe `Moyenne` vous est fournie, complète et correcte. Elle expose deux méthodes statiques.

Élément	Signature	Comportement
`calculer`	`public static double calculer(List<Integer>)`	La moyenne ; lève `IllegalArgumentException` si la liste est vide ou nulle, ou si une note sort de [0, 20]
`reussite`	`public static boolean reussite(List<Integer>)`	`true` si la moyenne atteint 10

Créez, dans `src/test/java` et à côté de la classe fournie, votre propre classe `be.ephec.java.jun.e03.MoyenneTest`. Elle doit compter au moins cinq méthodes annotées `@Test`, chacune portant un `@DisplayName` qui énonce le comportement vérifié.

Les cas à couvrir

Un test qui ne couvre que le cas où tout se passe bien ne protège pas : les défauts se logent dans les bords.

- le cas nominal — deux ou trois notes ordinaires ;
- une seule note ;

- une moyenne qui ne tombe pas juste ;
- les bornes acceptées — 0 et 20 ;
- la liste vide, et la liste nulle ;
- une note hors bornes, au-dessus comme en dessous ;
- la réussite exactement à 10, juste au-dessus et juste en dessous.

Pour comparer deux double, `assertEquals` demande un troisième argument, la tolérance : `assertEquals(14.0, moyenne, 0.001)`. Sans elle, deux calculs mathématiquement égaux peuvent différer au dernier chiffre.

Comment vérifier votre code

> `be.ephec.java.jun.e03.RedactionTest`
>
> Contrôle que votre classe existe, qu'elle compte au moins cinq méthodes `@Test`, que chacune porte un `@DisplayName`, et que leurs noms décrivent un comportement plutôt qu'un numéro.
>
> Il ne peut PAS vérifier que vos tests sont pertinents — aucun outil ne le peut. Le vert que vous obtiendrez ici ne dit rien de la qualité de ce que vous avez écrit. C'est précisément pourquoi `JUN.2` existait.

Questions à se poser

- Écrivez un test qui passe au vert sans rien vérifier. Combien de temps vous faut-il ? C'est aussi le temps qu'il faut pour se donner une fausse assurance.
- Modifiez temporairement `Moyenne` pour y introduire un défaut — un `>` au lieu d'un `>=`. Vos tests le détectent-ils ? C'est la seule mesure honnête de leur valeur.
- Combien de vos tests survivraient si la méthode changeait de nom ? Et si son comportement changeait ? La différence dit ce qu'un test protège vraiment.

Module GEN — Types génériques

Les exercices du module. Leurs codes sont **cliquables** : ils mènent directement à l'exercice.

Exercice	Package	Sujet
GEN.1	`be.ephec.java.gen.e01`	Une paire, un seul type
GEN.2	`be.ephec.java.gen.e02`	Un couple, deux types
GEN.3	`be.ephec.java.gen.e03`	Deux sortes de triplets
GEN.4	`be.ephec.java.gen.e04`	Le joker
GEN.5	`be.ephec.java.gen.e05`	Une méthode générique
GEN.6	`be.ephec.java.gen.e06`	Hériter d'une classe générique

Ce qu'il faut avoir en tête

Vous utilisez la généricité depuis le module `COL`, sans l'avoir écrite. C'est ce que signifient les chevrons de `List<Integer>` ou `Map<String, Fiche>` : la classe est écrite une fois, le type est décidé à l'utilisation.

```
public class Paire<T> {        // T : parametre de type
    private final T premier;   // s'utilise comme n'importe quel type
}

Paire<Integer> notes = new Paire<>(12, 16);   // T vaut Integer ici
Paire<String>  mots  = new Paire<>("a", "b"); // et String la
```

Ce que cela apporte. Sans généricité, il faudrait une classe par type, ou une classe d'`Object` avec un transtypage à chaque lecture. Le transtypage n'est pas seulement pénible : il déplace l'erreur de la compilation vers l'exécution.

```
Paire paire = new Paire(12, "seize");            // compile !
int somme = (Integer) paire.getSecond() + 1;      // plante a l'execution

Paire<Integer> paire = new Paire<>(12, "seize");  // ne compile pas
```

Une erreur détectée à la compilation coûte quelques secondes ; la même détectée en production coûte une soirée.

Une convention de nommage. Les lettres n'ont aucune importance pour le compilateur, mais l'usage veut `T` pour un type quelconque, `K` et `V` pour une clé et une valeur, `E` pour l'élément d'une collection.

GEN.1 — Une paire, un seul type

Package : ***`be.ephec.java.gen.e01`***

Objectif

Écrire sa première classe générique.

Énoncé

Complétez la classe `Paire<T>`, qui modélise une paire de deux objets de même type.

Élément	Signature	Rôle
`constructeur`	`public Paire(T premier, T second)`	Les deux valeurs, immuables
`getPremier`	`public T getPremier()`	La première

Élément	Signature	Rôle
`getSecond`	`public T getSecond()`	La seconde
`toString`	`public String toString()`	Format imposé, ci-dessous

```
premiere valeur : 12 - deuxieme valeur : 16
```

Écrivez ensuite le `Programme` : une paire d'`Integer`, une paire de `Double`, affichées.

L'expérience à faire

Essayez `new Paire<Integer>(12, "seize")` et lisez ce que dit le compilateur. Cette erreur — refusée avant même que le programme ne démarre — est exactement le service que rend la généricité.

Comment vérifier votre code

> `be.ephec.java.gen.e01.PaireTest`
>
> Vérifie une paire d'`Integer`, une paire de `String`, le format du `toString`, et le fait qu'aucun transtypage ne soit nécessaire pour additionner les deux valeurs.

Questions à se poser

- Le losange vide du `new Paire<>(12, 16)` : d'où le compilateur déduit-il qu'il s'agit d'`Integer` ?
- Peut-on écrire `Paire<int>` ? Essayez, et retenez la réponse : elle explique pourquoi on manipule des `Integer` et non des `int` dans les collections.

GEN.2 — Un couple, deux types

Package : ***`be.ephec.java.gen.e02`***

Objectif

Déclarer plusieurs paramètres de type.

Énoncé

Complétez `Couple2<U, V>`, qui modélise un couple de deux objets de types possiblement différents. Mêmes méthodes qu'en `GEN.1`, même format de `toString`.

Le `Programme` crée un couple de deux `Integer`, puis un couple contenant un `Integer` et un `Double`.

Notez le premier des deux cas. Deux paramètres de type *autorisent* la différence, ils ne l'*imposent* pas : `Couple2<Integer, Integer>` est parfaitement valide.

Comment vérifier votre code

> `be.ephec.java.gen.e02.Couple2Test`
>
> Vérifie deux types différents, deux types identiques, la combinaison `Integer` et `Double`, et le format du `toString`.

Questions à se poser

- Si `Couple2<T, T>` fait le travail de `Paire<T>`, pourquoi garder les deux classes ? Réfléchissez à ce qu'un type *dit*, en plus de ce qu'il stocke.

GEN.3 — Deux sortes de triplets

Package : `be.ephec.java.gen.e03`

Objectif

Refaire le geste sur trois éléments, et comparer les deux approches.

Énoncé

Complétez `TripletI<T>` — trois éléments de même type — et `TripletD<T, U, V>` — trois éléments de types possiblement différents. Chacune expose son constructeur et trois accesseurs.

Le `Programme` crée un `TripletI` de `String`, un d'`Integer`, puis un `TripletD` contenant un `String`, un `Integer` et un `Double`.

Comment vérifier votre code

> `be.ephec.java.gen.e03.TripletTest`
> Vérifie les deux classes, y compris le cas d'un `TripletD` dont les trois types coïncident.

Questions à se poser

- `TripletD` est strictement plus permissive que `TripletI`. Faut-il pour autant se passer de la seconde ? Que perdrait-on ?
- Combien de paramètres de type avant que la déclaration ne devienne illisible ? À partir de quel moment vaudrait-il mieux écrire une vraie classe, avec des noms de champs ?

GEN.4 — Le joker

Package : `be.ephec.java.gen.e04`

Objectif

Rencontrer la règle la plus déroutante de la généricité, et comprendre pourquoi elle existe.

Contexte

Le package fournit `Personne`, `Etudiant` qui en hérite, et un `Triplet<T>` complet.

Vous devez écrire une méthode capable de traiter aussi bien un triplet de personnes qu'un triplet d'étudiants. Commencez par la version évidente :

```
public static String noms(Triplet<Personne> triplet) { ... }
```

Elle compile. Puis essayez de lui passer un `Triplet<Etudiant>` — et lisez le refus du compilateur.

> **Un Etudiant est une Personne, mais un `Triplet<Etudiant>` n'est pas un `Triplet<Personne>`**
> La règle surprend tout le monde, et elle a une bonne raison. Si la substitution était permise, la méthode pourrait ranger une Personne ordinaire dans un triplet que son propriétaire croit rempli d'Etudiants. Le problème n'apparaîtrait qu'à la lecture suivante, ailleurs, sans rapport apparent.
> Java l'interdit donc à la compilation. Les tableaux, eux, l'autorisent — et le paient d'une `ArrayStoreException` à l'exécution.

Énoncé

Complétez `Affichage.noms(Triplet<? extends Personne>)`, qui retourne les trois noms séparés par `", "`. Le joker se lit « un triplet de quelque chose qui est une `Personne` ».

```
noms(triplet)   ->   "Ada, Alan, Grace"
```

On peut lire un tel triplet — tout ce qu'il contient est au moins une `Personne`. On ne peut rien y écrire, et c'est précisément ce qui rend la lecture sûre.

Comment vérifier votre code

> `be.ephec.java.gen.e04.AffichageTest`
>
> Vérifie que la même méthode accepte un `Triplet<Personne>`, un `Triplet<Etudiant>`, et un `Triplet<Personne>` contenant un mélange des deux.

Questions à se poser

- Dans `noms`, essayez d'écrire quelque chose dans le triplet reçu. Que dit le compilateur, et pourquoi a-t-il raison ?
- Un `Triplet<Personne>` peut contenir un `Etudiant`, mais un `Triplet<Etudiant>` n'est pas un `Triplet<Personne>`. Ces deux phrases se contredisent-elles ?

GEN.5 — Une méthode générique

Package : ***`be.ephec.java.gen.e05`***

Objectif

Déclarer un paramètre de type sur une méthode plutôt que sur une classe.

Contexte

Jusqu'ici, `T` appartenait à la classe. Il peut aussi n'appartenir qu'à une méthode : il se déclare alors juste avant le type de retour.

```
public static <T> String otGnirts(T objet) { ... }
           ^^^  la methode est generique, la classe ne l'est pas
```

Énoncé

Complétez `UtilGenerique.otGnirts`, qui retourne le `toString` de l'objet, à l'envers — d'où son nom incongru.

```
otGnirts("Java")   ->  "avaJ"
otGnirts(1234)     ->  "4321"
otGnirts(-12)      ->  "21-"
```

Indice : `StringBuilder` possède une méthode `reverse()`.

Le `Programme` l'appelle sur un `String`, un `Integer`, un `Double`, et sur un objet d'une classe que vous écrivez avec son propre `toString`.

Une remarque honnête

Cette méthode ne fait rien de `T` sinon appeler son `toString`. Elle aurait donc pu prendre un `Object` et fonctionner tout autant : la généricité n'apporte ici presque rien.

Elle prend tout son sens quand le type de retour dépend de `T` :

```
public static <T> T premier(List<T> liste)
```

L'appelant récupère alors un T, pas un `Object`, et n'a aucun transtypage à écrire. C'est là que la différence se voit.

Comment vérifier votre code

> `be.ephec.java.gen.e05.UtilGeneriqueTest`
>
> Vérifie sur une chaîne — y compris vide et d'un caractère —, sur un entier positif et négatif, sur un décimal, et sur un objet quelconque muni d'un `toString`.

Questions à se poser

- Réécrivez la méthode avec `Object` en paramètre. Le test passe-t-il toujours ? Qu'avez-vous perdu ?
- Que se passe-t-il si l'on passe `null` ? Comparez `objet.toString()` et `String.valueOf(objet)`.

GEN.6 — Hériter d'une classe générique

Package : ***`be.ephec.java.gen.e06`***

Objectif

Combiner héritage et généricité, et distinguer deux mots-clés qu'on confond souvent.

Énoncé

La classe `Couple<T>` vous est fournie. Écrivez `CoupleNomme<T>`, qui en hérite et connaît en plus un nom.

Élément	Signature	Rôle
`constructeur`	`public CoupleNomme(String nom, T premier, T second)`	Le `nom`, puis les deux valeurs
`getNom`	`public String getNom()`	Le `nom`
`description`	`public String description()`	Redéfinie pour inclure le `nom`

```
Couple       ->  "premier: 12, second: 16"
CoupleNomme  ->  "notes -> premier: 12, second: 16"
```

> **Deux mots-clés à ne pas confondre**
>
> `super(...)` appelle un constructeur de la classe MÈRE, et doit être la première instruction du constructeur. `this(...)` appelle un autre constructeur de la MÊME classe.
>
> Le même mot `super` sert aussi à appeler une méthode de la classe mère : `super.description()` vous évite de recopier son contenu. Recopier fonctionnerait — jusqu'au jour où la description de Couple change et où personne ne pense à répercuter. Voir `CC-CLS.3`.

Pourquoi description et affiche sont séparées

`Couple` expose une méthode `description()` qui fabrique le texte, et une méthode `affiche()` qui l'envoie à l'écran. C'est ce qui permet de tester la première — on ne vérifie pas ce qui est parti dans la console. Voir `CC-MET.7` du mémento Clean Code.

Comment vérifier votre code

> `be.ephec.java.gen.e06.CoupleNommeTest`
>
> Vérifie l'héritage, les accesseurs hérités, la description redéfinie, celle de la classe mère restée intacte, et surtout le polymorphisme : vue comme un Couple, l'instance retourne quand même sa propre description.

Questions à se poser

- `affiche()` n'est pas redéfinie, et pourtant elle affiche le nom pour un `CoupleNomme`. Comment est-ce possible ? Comparez avec la classe abstraite du module `ABS` : la classe mère y appelait déjà du code écrit après elle.
- Le `T` de `CoupleNomme<T>` et celui de `Couple<T>` sont-ils le même ? Que se passerait-il si l'on écrivait `class CoupleNomme<T> extends Couple<String>` ?

Module LAM — Lambdas et références de méthodes

Les exercices du module. Leurs codes sont **cliquables** : ils mènent directement à l'exercice.

Exercice	Package	Sujet
LAM.1	`be.ephec.java.lam.e01`	Un tableau de comportements
LAM.2	`be.ephec.java.lam.e02`	Écrire son interface fonctionnelle
LAM.3	`be.ephec.java.lam.e03`	Celle du JDK, et les références de méthodes

Ce qui change, et ce qui ne change pas

Vous savez déjà tout faire. En `ITF.1`, une méthode recevait un `Calculateur` en paramètre et déclenchait un calcul sans savoir lequel. En `COL.5`, vous avez écrit une classe `EstPair` pour la passer à `removeIf`.

Ce module n'ajoute aucune capacité. Il ajoute une écriture — et une écriture qui change tout, parce qu'elle rend praticable ce qui était seulement possible.

```
new Calculateur() {                         // classe anonyme
    public int calcul(int n) { return n * n; }
}

(int n) -> { return n * n; }                // lambda complete

n -> n * n                                  // lambda abregee
```

Les trois produisent exactement le même objet. On peut omettre le type du paramètre, les parenthèses s'il est unique, les accolades et le `return` si le corps tient en une expression.

Une seule condition

Une lambda ne s'écrit que pour une interface *fonctionnelle* : une interface qui ne déclare qu'UNE méthode abstraite. Sans cela, le compilateur ne saurait pas quelle méthode vous êtes en train d'implémenter.

L'annotation `@FunctionalInterface` n'est pas obligatoire, mais elle demande au compilateur de vérifier cette règle. Si quelqu'un ajoute une seconde méthode plus tard, il le signalera — au lieu de laisser toutes les lambdas cesser de compiler sans explication.

LAM.1 — Un tableau de comportements

Package : `be.ephec.java.lam.e01`

Objectif

Écrire ses premières lambdas, et ranger des comportements dans un tableau.

Énoncé

L'interface `Calculateur` vous est fournie : elle déclare `int calcul(int n)` et rien d'autre. Complétez la classe `Calculs`.

Élément	Signature	Rôle
`traite`	`public static int traite(int, Calculateur)`	Applique le `Calculateur` au nombre
`calculateurs`	`public static Calculateur[] calculateurs()`	Quatre lambdas : carré, double, opposé, inchangé

Élément	Signature	Rôle
`tousLesCalculs`	`public static String tousLesCalculs(int)`	Les quatre résultats, séparés par une espace

```
tousLesCalculs(3)    ->  "9 6 -3 3"
tousLesCalculs(-2)   ->  "4 -4 2 -2"
```

Le point de l'exercice

Un tableau de calculateurs. On y range des comportements comme on rangerait des nombres, et le programme les applique sans savoir ce qu'ils font. Comparez avec `ITF.1` : la même idée demandait quatre fichiers, elle tient ici en quatre lignes.

Comment vérifier votre code

> `be.ephec.java.lam.e01.CalculsTest`
>
> Vérifie la délégation, le contenu et l'ordre du tableau, les résultats combinés, et le fait qu'une lambda écrite dans le test — un cube, absent du tableau — fonctionne tout autant.

Questions à se poser

- Écrivez `n -> n * n` sous ses trois formes, de la classe anonyme à la lambda abrégée. Laquelle préférez-vous relire ? Laquelle préférez-vous écrire ?
- Ajoutez une seconde méthode à l'interface `Calculateur`. Que dit le compilateur, et sur quelle ligne ?
- Une lambda n'a pas de nom, et pourtant elle fait quelque chose. Comment le lecteur sait-il quoi ? Voir `CC-MET.1` : quand le nom disparaît, c'est la variable qui la reçoit qui doit parler.

LAM.2 — Écrire son interface fonctionnelle

Package : ***`be.ephec.java.lam.e02`***

Objectif

Faire passer un critère de sélection en paramètre.

Énoncé

Complétez l'interface fonctionnelle `Filtrage` — une seule méthode, `boolean test(int valeur)` — puis la méthode `Affichage.selectionner(int[], Filtrage)`, qui retourne les éléments retenus séparés par une espace.

```
int[] tableau = {1, 4, -9, 5, -7, -3, 2};

selectionner(tableau, e -> e > 0)    ->  "1 4 5 2"
selectionner(tableau, e -> e < 0)    ->  "-9 -7 -3"
selectionner(tableau, e -> e > 1000) ->  ""
```

Le `Programme` affiche les positifs, en passant le critère sous forme d'expression lambda.

La question à garder

Cherchez, parmi les interfaces fonctionnelles du JDK vues au cours, celle qui ferait exactement le travail de votre `Filtrage`. Elle vit dans `java.util.function`. `LAM.3` s'en sert, et votre interface disparaît.

Comment vérifier votre code

> `be.ephec.java.lam.e02.AffichageTest`
>
> Vérifie quatre filtres différents — dont un qui ne retient rien et un qui retient tout — et le cas du tableau vide.
>
> Remarquez que le test passe des lambdas que la méthode n'a jamais vues. C'est le même service qu'en `ITF.1` : `selectionner` ne connaît que l'interface.

Questions à se poser

- Le séparateur ne doit apparaître ni avant le premier élément ni après le dernier. Vous aviez déjà résolu ce problème en `TAB.1` : la solution est-elle la même ?
- Une lambda peut-elle utiliser une variable locale déclarée avant elle ? Essayez, puis essayez de modifier cette variable ensuite.
- Une lambda qui modifie quelque chose au passage est légale, et c'est un piège. Voir `CC-MET.7`.

LAM.3 — Celle du JDK, et les références de méthodes

Package : ***`be.ephec.java.lam.e03`***

Objectif

Découvrir que le JDK avait déjà l'interface, et donner un nom à ses critères.

Énoncé — première partie

`Filtrage` était un doublon. `java.util.function.IntPredicate` déclare exactement `boolean test(int)`. Reprenez `selectionner` en l'utilisant : rien d'autre ne change.

Ce n'était pas du temps perdu. En écrivant votre propre interface, vous avez compris ce qu'est une interface fonctionnelle. Mais avant d'en écrire une, regardez désormais ce que contient `java.util.function` : `Predicate`, `Function`, `Supplier`, `Consumer` et leurs variantes primitives couvrent la plupart des besoins.

Énoncé — seconde partie

Écrivez trois méthodes statiques — `estPositif`, `estNegatif`, `estPair` — chacune sur une ligne, avec la signature qu'attend `IntPredicate`. Puis la méthode `rapport`, qui les passe par **référence de méthode** :

```
Affichage::estPositif          au lieu de       e -> e > 0
```

Le rapport tient en trois lignes, sans retour à la ligne final :

```
Les positifs: 1 4 5 2
Les negatifs: -9 -7 -3
Les pairs: 4 2
```

> **Lambda ou référence de méthode ?**
>
> Les deux produisent le même objet, et sont interchangeables — le test le vérifie explicitement.
>
> La lambda est anonyme et s'écrit sur place : parfaite pour un critère utilisé une fois. La référence donne un nom au critère, ce qui le rend réutilisable et testable séparément — la classe de test vérifie d'ailleurs les trois critères isolément avant de vérifier le rapport.
>
> Le nom est aussi une explication. Comparez « `e -> e % 2 == 0` » et « `estPair` » dans une expression un peu longue. Voir `CC-NOM.1`.

Comment vérifier votre code

> `be.ephec.java.lam.e03.AffichageTest`
>
> Vérifie les trois critères isolément — y compris zéro, qui n'est ni positif ni négatif mais bien pair —, l'équivalence entre lambda et référence de méthode, et le format du rapport.

Questions à se poser

- Reprenez `COL.5` et remplacez la classe `EstPair` par une lambda, puis par une référence de méthode. Combien de lignes disparaissent ?
- Une référence de méthode s'écrit aussi sur une instance — `monObjet::maMethode` — et sur un constructeur — `Etudiant::new`. Cherchez dans quel contexte cette dernière forme sert.
- Vos trois critères sont des méthodes statiques. Auraient-ils pu être des méthodes d'instance ? Qu'est-ce qui change à l'appel ?
- Comparez la version avec boucle et la version avec stream. Laquelle dit le plus clairement ce qu'elle fait ? Voir `CC-MET.3`.

Module FX — Interfaces graphiques avec JavaFX

Les exercices du module. Leurs codes sont **cliquables** : ils mènent directement à l'exercice.

Exercice	Package	Sujet
FX.1	be.ephec.java.fx.e01	Un compteur de participants
FX.2	be.ephec.java.fx.e02	Un bouton Reset
FX.3	be.ephec.java.fx.e03	Deux compteurs, et une mise en page
FX.4	be.ephec.java.fx.e04	Compter au clavier
FX.5	be.ephec.java.fx.e05	Un événement voyage
FX.6	be.ephec.java.fx.e06	Écouter une propriété
FX.7	be.ephec.java.fx.e07	Un Slider et un Label
FX.8	be.ephec.java.fx.e08	Lier plutôt qu'écouter
FX.9	be.ephec.java.fx.e09	Une calculatrice, en MVC

La classe à lancer

Chaque exercice contient une classe `Lanceur`. **C'est cette classe-là qu'il faut exécuter**, pas celle qui hérite d'`Application`.

Lancer l'autre donne un message déroutant — « des composants d'exécution `JavaFX` obligatoires sont manquants » — alors que tout est pourtant installé. L'explication tient en une page : `TEC-FX.4` du mémento technique.

Vous n'avez rien à écrire dans cette classe, et rien à y comprendre pour faire les exercices. Lisez la page le jour où la curiosité vous prend, ou celui où vous rencontrerez le message dans un projet à vous.

Ce module n'a aucune classe de test

C'est le seul du recueil dans ce cas, et ce n'est pas un oubli. On ne vérifie pas une fenêtre par une assertion : il faudrait simuler des clics, attendre que l'affichage se rafraîchisse, et l'on finirait par tester `JavaFX` plutôt que votre code.

La vérification vous revient donc. Chaque énoncé se termine par une liste de ce que vous devez *observer* en lançant l'application. Parcourez-la point par point : c'est elle qui remplace le vert de la barre `JUnit`.

Une exception : le `Modele` de `FX.9` ne connaît rien à `JavaFX`, et se teste comme n'importe quelle classe. C'est précisément ce que l'architecture MVC permet de gagner.

Le vocabulaire, du plus grand au plus petit

Terme	Ce que c'est
`Stage`	la fenêtre, fournie par le système
`Scene`	ce que la fenêtre affiche
`racine`	le composant unique que la scène contient
`conteneur`	un composant qui en contient d'autres — `HBox`, `VBox`, `BorderPane`
`contrôle`	un composant avec lequel on interagit — `Button`, `TextField`, `Slider`

```
public class MonApp extends Application {
    public static void main(String[] args) { launch(args); }

    @Override
    public void start(Stage stage) {
        Button bouton = new Button("Cliquez");
        bouton.setOnAction(evenement -> jeFaisQuelqueChose());

        stage.setScene(new Scene(new HBox(bouton)));
        stage.setTitle("Mon application");
        stage.show();
    }
}
```

Deux réflexes à prendre tout de suite

Ce qui doit survivre à start est un champ. Une variable locale de `start` disparaît à la fin de la méthode, alors que la fenêtre, elle, reste ouverte. Tout composant que vous devrez relire ou modifier plus tard est donc un champ de la classe.

La lambda dit quand, la méthode dit quoi. Écrivez le traitement dans une méthode nommée, et faites-la appeler par la lambda. Vous gagnez un nom, et la possibilité d'appeler le même traitement depuis ailleurs — ce dont `FX.4` aura besoin. Voir `CC-MET.2`.

Si l'application ne démarre pas, voir `ERR.7` du guide IntelliJ — `"JavaFX runtime components are missing"`. C'est le message le plus fréquent des premières minutes, et il n'a rien à voir avec votre code.

FX.1 — Un compteur de participants

Package : ***`be.ephec.java.fx.e01`***

Objectif

Écrire sa première application, et brancher un bouton.

Énoncé

Créez une application composée d'un champ texte et d'un bouton. Elle vous permettra de compter les participants à un événement : à chaque participant, vous enfoncerez le bouton « +1 » et la valeur du champ texte sera incrémentée. La valeur initiale est « 0 ».

Élément	Rôle
`TextField`	affiche le compte, en lecture seule
`Button « +1 »`	ajoute un participant
`HBox`	range les deux côte à côte

Ce que vous devez observer

- la fenêtre s'ouvre, porte un titre, et affiche « 0 » ;
- chaque clic augmente le nombre de un, sans jamais sauter ni revenir en arrière ;
- on ne peut pas entrer de données à la main dans le champ texte (il est en lecture seule).

Questions à se poser

- Déclarez le compteur en variable locale de `start` plutôt qu'en champ. Que dit le compilateur, et que vous apprend son message ?
- Le champ texte est en lecture seule. Rien ne l'imposait : pourquoi est-ce quand même la bonne décision ?
- Que se passe-t-il si vous oubliez `stage.show()` ? Le programme plante-t-il ?

FX.2 — Un bouton Reset

Package : ***`be.ephec.java.fx.e02`***

Objectif

Deux gestionnaires sur deux composants, et la première duplication à éviter.

Énoncé

Reprenez l'application précédente en lui ajoutant un bouton « Reset » qui remet le compteur à zéro.

> **Le squelette vous impose une troisième méthode**
>
> `incrementer()` et `remettreAZero()` finissent toutes deux par la même chose : écrire le compteur dans le champ. Le squelette déclare `rafraichir()` pour que cette ligne ne soit écrite qu'une fois.
>
> Ce n'est pas une coquetterie. Au prochain exercice, vous aurez deux champs, et cette ligne en deviendra deux. Le jour où l'affichage change, vous voudrez avoir un seul endroit à modifier. Voir `CC-CLS.3`.

Ce que vous devez observer

- « +1 » continue de fonctionner comme en `FX.1` ;
- « Reset » ramène l'affichage à « 0 », quel que soit le compte atteint ;
- après un Reset, « +1 » repart bien de 1 et non du nombre précédent.

Questions à se poser

- Les deux boutons ne se connaissent pas. Où est écrit le lien qui fait qu'ils agissent sur le même compteur ?

FX.3 — Deux compteurs, et une mise en page

Package : ***`be.ephec.java.fx.e03`***

Objectif

Composer des conteneurs pour obtenir une disposition à deux colonnes.

Énoncé

Votre application fonctionnant bien, on vous demande maintenant de compter séparément les hommes et les femmes. Ajoutez donc un deuxième bouton et un deuxième champ texte, ainsi qu'un libellé au-dessus de chaque compteur. Le bouton « Reset » remet les deux à zéro.

La difficulté n'est plus l'événement, c'est l'agencement. Un `HBox` range ses enfants côte à côte, un `VBox` les empile, et on les imbrique :

```
VBox
 +- HBox          les deux colonnes
 |   +- VBox      libelle + champ + bouton, hommes
 |   +- VBox      libelle + champ + bouton, femmes
 +- Button        Reset
```

Pour l'espacement : `new VBox(10, a, b)` met dix pixels entre les enfants, et `setStyle("-fx-padding: 15;")` donne une marge intérieure au conteneur.

Ce que vous devez observer

- deux colonnes côte à côte, chacune surmontée de son libellé ;
- chaque bouton « +1 » n'agit que sur son propre compteur ;
- « Reset » remet les deux à zéro en une fois ;
- la fenêtre reste lisible si on l'agrandit.

Questions à se poser

- Les deux colonnes sont identiques à trois choses près. Pouvez-vous les fabriquer par une seule méthode qui prend ces trois choses en paramètres ?
- Cette méthode retournerait un `VBox` au lieu de modifier un état. Pourquoi est-ce préférable ? Voir `CC-MET.7`.

FX.4 — Compter au clavier

Package : `be.ephec.java.fx.e04`

Objectif

Traiter un événement qui n'appartient à aucun composant.

Énoncé

Cliquer sur les boutons n'est pas très pratique quand les gens défilent. Faites en sorte que les compteurs s'incrémentent quand on enfonce la touche 1 pour le premier et 2 pour le second, et qu'une pression sur 0 remette les deux à zéro.

Les boutons continuent de fonctionner : on ajoute une façon de faire, on n'en retire pas.

> **Le point délicat**
>
> Un événement clavier ne se pose pas sur un bouton : une touche frappée n'appartient à aucun composant en particulier. Posez le gestionnaire sur la racine.
>
> `racine.addEventHandler(KeyEvent.KEY_TYPED, evenement -> ...)`
>
> `KEY_TYPED` donne le caractère frappé, par `getCharacter()`. C'est une `String`, pas un char.

Ce que vous devez observer

- les touches 1, 2 et 0 agissent comme les boutons correspondants ;
- les boutons fonctionnent toujours ;
- toute autre touche ne provoque rien — ni erreur, ni message, ni gel de la fenêtre ;
- cela marche que vous ayez cliqué sur un bouton juste avant ou non.

Questions à se poser

- Pourquoi poser le gestionnaire sur la racine plutôt que sur un champ texte fonctionne-t-il même quand un bouton a le focus ? La réponse est le sujet de l'exercice suivant.
- Cherchez la différence entre `KEY_TYPED` et `KEY_PRESSED`. Laquelle utiliseriez-vous pour réagir à la touche « flèche haut » ?

FX.5 — Un événement voyage

Package : `be.ephec.java.fx.e05`

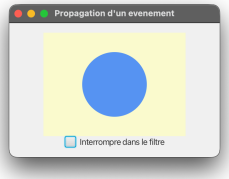
Objectif

Constater qu'un événement traverse toute la hiérarchie, deux fois.

Énoncé — première partie

Créez un `Stage` contenant l'empilement suivant, de l'extérieur vers l'intérieur :

```
Stage
 +- Scene
     +- BorderPane
         +- StackPane
             +- Rectangle
             +- Circle
```



Posez un `EventFilter` sur le `BorderPane` et **deux** `EventHandler` sur le `Circle`, tous pour `MouseEvent.MOUSE_CLICKED`. Chacun affiche une ligne sur la console.

Cliquez sur le cercle. Les trois s'affichent. Notez l'ordre : il n'est pas celui qu'on attend.

Énoncé — seconde partie

Modifiez votre code pour que les deux gestionnaires du cercle ne réagissent plus — sans les supprimer, et sans toucher au cercle. Une seule ligne suffit, dans le filtre.

Ce que vous devez observer

- le filtre du conteneur s'affiche AVANT les deux gestionnaires du cercle ;
- les deux gestionnaires s'exécutent tous les deux, dans l'ordre où ils ont été posés ;
- un clic sur le rectangle, à côté du cercle, ne déclenche que le filtre ;
- après la seconde partie, plus aucune ligne du cercle ne s'affiche.

Questions à se poser

- Filtre et gestionnaire portent deux noms parce qu'ils interviennent à deux moments. Lesquels, et dans quel sens l'événement va-t-il à chaque fois ?
- Deux gestionnaires sur le même événement s'exécutent tous les deux. En quoi est-ce différent de `setOnMouseClicked`, appelé deux fois ?
- Citez un cas réel où l'on veut qu'un conteneur empêche ses enfants de réagir.

FX.6 — Écouter une propriété

Package : `be.ephec.java.fx.e06`

Objectif

Réagir à un changement de valeur, ce qui n'est pas réagir à un événement.

Énoncé

Cherchez dans la Javadoc de `TextField` : a-t-elle des propriétés ? Trouvez celle à laquelle il faut ajouter un Listener pour afficher les valeurs successives encodées dans le champ. Écrivez l'application qui met cela en œuvre.

> **La démarche compte autant que le résultat**
>
> Ne cherchez pas la réponse ailleurs que dans l'éditeur. Ctrl+Q sur le type ouvre sa Javadoc ; la complétion vous donne la liste de ses méthodes.
>
> Une propriété se reconnaît à son accesseur : une méthode dont le nom finit par Property et qui retourne un objet, non une valeur. C'est exactement la démarche de la démonstration `TMP`.

Ce que vous devez observer

- chaque caractère tapé produit une ligne sur la console ;
- effacer un caractère en produit une aussi ;
- coller du texte avec `Ctrl+V` en produit une seule, avec toute la valeur — et c'est là que cette approche bat un gestionnaire de touches ;
- l'ancienne et la nouvelle valeur sont toutes deux disponibles.

Questions à se poser

- Le Listener reçoit trois arguments. À quoi sert le premier, que l'on ignore presque toujours ?
- Un gestionnaire sur `KEY_TYPED` aurait-il détecté un collage ? Et une modification faite par le programme lui-même ?

FX.7 — Un Slider et un Label

Package : `be.ephec.java.fx.e07`

Objectif

Refaire le geste précédent sur une valeur numérique, et rencontrer deux pièges.

Énoncé

Le `Slider` présente une `valueProperty` liée à la valeur du curseur. Écrivez une application dans laquelle un `Label` se met à jour en fonction de cette valeur, à l'aide d'un `ChangeListener`.

Le piège	Ce qu'il faut faire
le Listener reçoit un `Number`, pas un double	`nouvelleValeur.doubleValue()`
un double brut s'affiche avec quinze décimales	`String.format("%1$,.2f", valeur)`
un Listener ne se déclenche qu'aux changements	afficher la valeur initiale avant l'ouverture

Une remarque sur le formatage. Tous les exercices depuis `TAB.1` imposaient `Locale.US` pour que le résultat soit prévisible et donc testable. Ici, c'est un humain qui lit : laissez la locale de la machine faire son travail. Le même réflexe donnerait la mauvaise réponse.

Ce que vous devez observer

- le Label affiche une valeur dès l'ouverture, sans qu'on ait touché au curseur ;
- la valeur suit le curseur pendant le glissement, pas seulement au relâchement ;
- deux décimales, jamais quinze ;
- les bornes 0 et 100 s'affichent correctement.

Questions à se poser

- Retirez l'appel qui affiche la valeur initiale. Que voit l'utilisateur en ouvrant l'application, et pourquoi est-ce gênant ?
- Pourquoi le Listener reçoit-il un `Number` alors que la propriété est une `DoubleProperty` ?

FX.8 — Lier plutôt qu'écouter

Package : `be.ephec.java.fx.e08`

Objectif

Déclarer qu'une valeur dépend d'une autre, au lieu d'écrire le code qui la recopie.

Énoncé

Écrivez une fenêtre composée d'un champ de saisie pour un pseudo et de trois composants qui en dépendent. **Aucun Listener, aucun gestionnaire de clavier** : chaque dépendance s'écrit en une instruction, par `bind`.

Composant	Ce qu'il affiche ou fait
`Label compteur`	`"5 / 12 caracteres"`, la longueur du pseudo sur la longueur maximale
`Label salut`	`"Bonjour, Alice !"`, avec le pseudo tel qu'il est tapé
`Button « Valider »`	désactivé tant que le pseudo est vide ou dépasse douze caractères ; affiche le pseudo retenu sur la console

> **Les outils, tous sur la propriété du texte**
>
> `length()` et `isEmpty()` retournent une longueur et une condition observables, qui se recalculent seules.
>
> `asString("%d ...")` transforme un nombre observable en texte observable ; `greaterThan(n)` et `or(...)` fabriquent des conditions.
>
> `Bindings.concat(...)` assemble des morceaux de texte, observables ou non.

La longueur maximale est une constante nommée, déclarée dans le squelette, et le message l'affiche. Voir `CC-NOM.7`.

Ce que vous devez observer

- dès l'ouverture, le compteur affiche `"0 / 12 caracteres"` et le bouton est grisé, sans qu'aucune ligne de votre code ne l'ait demandé ;
- chaque caractère tapé, effacé ou collé met à jour le compteur et le salut ;
- le bouton devient actif au premier caractère, et redevient grisé au treizième ;
- un clic sur « Valider » affiche le pseudo sur la console.

Questions à se poser

- `FX.7` demandait d'afficher la valeur initiale avant l'ouverture. Pourquoi n'est-ce plus nécessaire ici ?
- Après le bind, appelez `compteur.setText("test")`. Que se passe-t-il, et pourquoi est-ce une protection plutôt qu'une gêne ?
- Récrivez `FX.7` avec un binding. Combien de lignes disparaissent ? Et que perdez-vous ?

FX.9 — Une calculatrice, en MVC

Package : `be.ephec.java.fx.e09`

Objectif

Construire une application complète, de l'écran au calcul, et séparer ce qui s'affiche de ce qui calcule.

> **Un exercice guidé**
>
> Cet exercice se fait en suivant le deck `FX3` — « La calculatrice, pas à pas ». Il construit la calculatrice étape par étape, dans l'ordre où vous l'écrivez : la vue d'abord, puis le contrôleur branché sur ses boutons, enfin le modèle et les opérateurs. Gardez le deck ouvert à côté de votre projet, et faites chaque étape au moment où il la montre.

Contexte : l'architecture MVC

Dès qu'une application dépasse quelques composants, on sépare trois responsabilités.

Rôle	Ce qu'il fait	Ici
`le Modèle`	les données et la logique métier	`Modele.java` — à écrire
`la Vue`	la description de l'écran	`calculatrice.fxml` — à écrire
`le Contrôleur`	écoute la vue, interroge le modèle, met la vue à jour	`Controleur.java` — à écrire

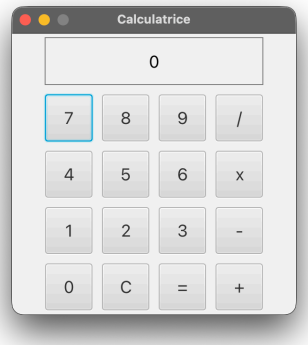
Le modèle ne connaît rien à `JavaFX`. Aucun `import`, aucune référence à un composant. C'est ce qui le rend testable : vous pourriez écrire une classe de test `JUnit` pour `calculer()`, alors qu'aucun test ne peut vérifier une fenêtre.

La vue, à écrire

Le squelette ne contient que la racine de la vue : une `VBox` qui nomme son contrôleur. Vous écrivez le reste en suivant la section 2 du deck `FX3` : l'écran, puis les quatre lignes de touches. Tout ce que le fichier contiendra remplace du code que vous auriez écrit à la main dans les huit exercices précédents.

La vue attendue

Élément	Contenu	Dimensions
`VBox (la racine)`	l'écran, puis quatre lignes de touches	300 × 300, espacement 10, centrée
`StackPane`	un Rectangle (le cadre) et le Text affichage	cadre de 230 × 50
`HBox × 4`	7 8 9 / — 4 5 6 x — 1 2 3 - — 0 C = +	espacement 10, centrées
`Button`	une touche	50 × 50



Trois attributs relient cette vue au contrôleur :

Dans le FXML	Dans le contrôleur
fx:`controller="...Controleur"`	désigne la classe qui reçoit les événements
fx:`id="affichage"`	`@FXML private` Text affichage — injecté au chargement
`onAction="#chiffrePresse"`	`@FXML private void chiffrePresse(ActionEvent)`

Énoncé

Complétez `Modele` et `Controleur`, après avoir écrit la vue. `ApplicationCalculatrice` et `Lanceur` vous sont fournis.

Méthode	Ce qu'elle fait
`Modele.calculer`	applique l'opérateur — "+", "-", "x" ou "/" — aux deux nombres
`chiffrePresse`	un seul gestionnaire pour les dix boutons de chiffres
`operateurPresse`	les opérateurs et le « = »
`effacer`	le bouton « C »

> **Un seul gestionnaire pour dix boutons**
>
> Le FXML branche les dix chiffres sur la même méthode. C'est l'événement qui dit lequel a été pressé :
>
> `String chiffre = ((Button) evenement.getSource()).getText();`
>
> Écrire dix méthodes aurait fonctionné, et aurait été dix fois la même chose.

La logique, une fois posée

Quand on presse un opérateur : s'il y avait déjà un opérateur en attente, on calcule ; on affiche le résultat ; on retient le nouvel opérateur. Le « = » suit le même chemin, sauf qu'il ne met rien en attente.

Le contrôleur doit donc retenir, entre deux clics, le premier nombre et l'opérateur choisi. Réfléchissez aussi au cas du tout premier opérateur pressé : il n'y a rien en attente.

Deux règles d'affichage imposées

- si l'écran montre « 0 », un chiffre le remplace ; sinon il s'y accole — sans quoi taper 5 puis 6 afficherait « 056 » ;
- après un opérateur, le chiffre suivant repart de zéro — sans quoi 5 + 6 afficherait « 56 ».

La division par zéro

La calculatrice affiche `Erreur` et se remet à zéro. Le modèle lève une exception plutôt que de retourner 0 : retourner 0 ferait passer une erreur pour un résultat, et le contrôleur continuerait à calculer avec une valeur fausse. Voir `CC-ERR.1`.

Ce que vous devez observer

- 12 + 5 = affiche 17 ; 12 − 5 = affiche 7 ; 12 x 5 = affiche 60 ;
- 13 / 4 = affiche 3, parce que le modèle travaille sur des entiers ;
- 1 / 0 = affiche « Erreur », et la calculatrice repart proprement ensuite ;
- taper 5 puis 6 affiche « 56 », et non « 056 » ;
- après 5 +, taper 6 affiche « 6 » et non « 56 » ;
- 2 + 3 + 4 = affiche 9 : chaque opérateur calcule ce qui précède ;
- « C » remet tout à zéro, y compris l'opérateur en attente.

Questions à se poser

- Écrivez une classe de test `JUnit` pour le Modèle. Combien de temps cela vous prend-il ? Et pour le contrôleur ?
- Le champ `affichage` vaut `null` tant que le FXML n'est pas chargé. Où pourriez-vous mettre du code qui s'exécute juste après l'injection ? Cherchez `initialize` dans la documentation de FXML.
- La vue ne connaît pas le modèle, et le modèle ne connaît pas la vue. Qui fait le lien, et que se passerait-il si on l'enlevait ?

Module FIC — Fichiers

Les exercices du module. Leurs codes sont **cliquables** : ils mènent directement à l'exercice.

Exercice	Package	Sujet
FIC.1	be.ephec.java.fic.e01	Trois classes qui se connaissent
FIC.2	be.ephec.java.fic.e02	Enregistrer des objets
FIC.3	be.ephec.java.fic.e03	Enregistrer du texte

Ce qu'il faut avoir en tête

Tout ce que votre programme construit disparaît quand il s'arrête. Écrire dans un fichier est la façon la plus simple de faire survivre une information — et il y a deux façons de s'y prendre, aux propriétés opposées.

	Sérialisation (.dat)	Texte (.txt)
Ce qu'on écrit	l'objet entier, tel quel	ce qu'on a décidé d'écrire
Qui peut le lire	un programme Java, et lui seul	n'importe qui, n'importe quoi
Si la classe change	les anciens fichiers deviennent illisibles	le fichier reste lisible
Effort d'écriture	presque nul	il faut choisir un format

Aucun des deux n'est meilleur. La question à se poser est : qui lira ce fichier, et quand ? `FIC.2` et `FIC.3` enregistrent exactement les mêmes étudiants, et vous comparerez les deux résultats.

Deux réflexes à prendre

Le try-avec-ressources. Un flux doit être fermé, y compris si une exception survient au milieu. Cette écriture s'en charge toute seule :

```
try (PrintWriter sortie = new PrintWriter(new FileWriter(fichier))) {
    sortie.println("...");
}   // ferme automatiquement le fichier, même en cas d'exception
```

Les exceptions d'entrée-sortie sont contrôlées. Un fichier peut être absent, verrouillé, plein. Le compilateur vous obligera donc à traiter ou à déclarer `IOException` — voir le tableau du module `EXC`.

FIC.1 — Trois classes qui se connaissent

Package : ***`be.ephec.java.fic.e01`***

Objectif

Construire le petit modèle qui servira aux deux exercices suivants. Aucun fichier ici.

Énoncé

Classe	Ce qu'elle contient	Méthodes
`Enseignant`	nom, prénom — au constructeur	les accesseurs, `toString`
`Cours`	intitulé au constructeur, chargé de cours modifiable	accesseurs, setter, `toString`
`Etudiant`	nom, prénom au constructeur ; la liste de cours démarre vide	`inscrire`, `desinscrire`, `getCours`, `nombreDeCours`, `toString`

Les formats de `toString` sont imposés :

```
Enseignant   ->  "Alan Turing"
Cours        ->  "Java (Alan Turing)"
Etudiant     ->  "Lovelace Ada : Java, Algebre"
             ->  "Lovelace Ada"          s'il n'a aucun cours
```

Une seconde inscription au même cours ne change rien ; désinscrire un cours absent ne provoque pas d'erreur.

> **Ce que retourne `getCours`**
>
> Un test vérifie qu'on ne peut pas vider l'étudiant à travers la liste qu'il retourne. Si vous retournez la liste elle-même, n'importe qui peut appeler `clear()` dessus et contourner `desinscrire` — l'encapsulation ne sert alors plus à rien.
>
> Cherchez `Collections.unmodifiableList` dans la Javadoc. Voir `CC-OBJ.2` du mémento Clean Code.

Comment vérifier votre code

> `be.ephec.java.fic.e01.EtudiantTest`
>
> Vérifie les trois classes, les formats de `toString`, l'inscription et la désinscription — y compris les cas sans effet — et la protection de la liste retournée.

FIC.2 — Enregistrer des objets

Package : ***`be.ephec.java.fic.e02`***

Objectif

Écrire des objets dans un fichier et les relire, sans écrire une ligne de format.

Contexte

Les trois classes de `FIC.1` vous sont fournies ici, complètes, et elles implémentent `Serializable`.

Cette interface ne déclare AUCUNE méthode. C'est un marqueur : une façon de dire à la JVM « cette classe accepte d'être transformée en octets ». Sans elle, l'écriture lèverait `NotSerializableException`.

Énoncé

Élément	Signature	Rôle
`enregistrer`	`public static void enregistrer(List<Etudiant>, File)`	Écrit la liste, en un seul `writeObject`
`relire`	`public static List<Etudiant> relire(File)`	Relit la liste

Les deux classes à connaître : `ObjectOutputStream` construit sur un `FileOutputStream` pour écrire, `ObjectInputStream` sur un `FileInputStream` pour relire.

Trois choses à observer

Un seul `writeObject` suffit pour toute la liste. La sérialisation suit les références : écrire la liste écrit les étudiants, donc leurs cours, donc leurs enseignants. C'est pourquoi les trois classes doivent être `Serializable`, et pas seulement celle qu'on nomme. Le test le vérifie.

`readObject` retourne un `Object`. Un transtypage est inévitable, et le compilateur avertit qu'il ne peut pas le vérifier. Ce n'est pas une erreur de votre part : la sérialisation est antérieure à la généricité, et le fichier ne contient aucune trace du type paramétré.

Ouvrez `etudiants.dat` dans un éditeur de texte (sans l'enregistrer). Vous y verrez quelques noms de classes noyés dans des octets illisibles. Le format est fait pour la JVM, pas pour vous. Gardez l'image : `FIC.3` prend l'autre parti.

Comment vérifier votre code

> `be.ephec.java.fic.e02.EnregistrementTest`
>
> Vérifie la création du fichier, l'aller-retour complet, la conservation des cours et de leurs enseignants, et le cas de la liste vide.
>
> Le test écrit dans un dossier temporaire fourni par `JUnit` — l'annotation `@TempDir` — créé avant le test et supprimé après. Vos tests ne laisseront jamais de fichiers dans le projet.

Questions à se poser

- Que se passe-t-il si relire reçoit un fichier qui n'existe pas ? Votre méthode doit-elle retourner `null`, une liste vide, ou lever une exception ? Voir `CC-ERR.1` et `CC-ERR.2`.
- Renommez un champ d'Etudiant, puis relisez un fichier écrit avant le changement. Que se passe-t-il ? Cherchez ce qu'est un `serialVersionUID`.
- Un mot de passe stocké dans un champ part-il dans le fichier ? Cherchez le mot-clé `transient`.
- Le message d'erreur que vous affichez en cas d'échec dit-il quel fichier a échoué ? Voir `CC-ERR.4`.

FIC.3 — Enregistrer du texte

Package : ***`be.ephec.java.fic.e03`***

Objectif

Enregistrer les mêmes données dans un format lisible, et mesurer ce qu'on y gagne et ce qu'on y perd.

Énoncé

Élément	Signature	Rôle
`ecrire`	`public static void ecrire(List<Etudiant>, File)`	Un étudiant par ligne
`lireLignes`	`public static List<String> lireLignes(File)`	Les lignes du fichier, telles quelles
`relire`	`public static List<Etudiant> relire(File)`	Reconstruit les étudiants

Le format est imposé : le nom, le prénom et les intitulés de cours séparés par des points-virgules ; les cours séparés entre eux par des virgules.

```
Lovelace;Ada;Java,Algebre
Turing;Alan;
```

Un piège à l'analyse. `"Turing;Alan;".split(";")` retourne un tableau de DEUX éléments, pas trois : `split` supprime les champs vides de fin. Le second argument `-1` le lui interdit. Le test contient un étudiant sans cours, précisément pour ça.

La comparaison

Les cours reconstruits n'ont plus de chargé de cours : le format texte ne l'a pas écrit. Créez donc un enseignant `"Inconnu Inconnu"` pour chacun — et mesurez ce que cela signifie.

La question de fond

Le fichier `.dat` conserve tout, mais ne se lit que par un programme Java, et seulement tant que les classes n'ont pas changé. Le fichier `.txt` ne conserve que ce qu'on a décidé d'écrire, mais s'ouvre dans n'importe quel éditeur, se traite par un tableur ou par un autre langage, et survit à toutes les évolutions de vos classes.

Savez-vous lire `etudiants.txt` ? Et `etudiants.dat` ? La réponse à ces deux questions est tout l'enjeu du choix d'un format.

Comment vérifier votre code

`be.ephec.java.fic.e03.EcritListeEtudiantsTest`

Vérifie le format d'écriture au caractère près, le cas de la liste vide, la reconstruction des étudiants — y compris celui sans cours — et le fait que le chargé de cours ait bel et bien été perdu.

Questions à se poser

- Que se passe-t-il si le nom d'un étudiant contient un point-virgule ? Comment les formats réels — `CSV`, par exemple — traitent-ils ce problème ?
- Pour conserver le chargé de cours dans le fichier texte, que faudrait-il ajouter au format ? À partir de quel moment vaudrait-il mieux utiliser un format existant comme `JSON` ?
- Le fichier écrit dépend du dossier depuis lequel on lance le programme. Où atterrit `new File("etudiants.txt")` quand on lance depuis IntelliJ ? Affichez `fichier.getAbsolutePath()` pour le savoir.
- La méthode qui écrit et celle qui met en forme sont-elles séparées ? Voir `CC-MET.3`.